

Tenstorrent Blackhole における FP32/FP64 相当行列積

RISC-V ベースの AI プロセッサ上での高精度行列積

棕木 大地

名古屋大学

第 1 部：背景

- ・ AI アクセラレータと FP64 のギャップ
- ・ Tenstorrent Blackhole とは

第 2 部：素朴な方法と、その壁

- ・ 無誤差分割 (3xTF32 と同型)
- ・ なぜ 13 bit で止まるのか

第 3 部：尾崎スキーム入門 (手法を知らない方向け)

- ・ スキーム I：整数の「桁」に割る
- ・ スキーム II：「余り」だけで計算する

第 4 部：Blackhole への実装

- ・ 完全オンデバイス
- ・ FP64 が無い機械での再構成
- ・ 各データパスへの具体化

第 5 部：評価

- ・ 全方式の統一比較
- ・ どこがボトルネックか

第 6 部：考察とハードウェアへの提言

第 1 部：背景

深層学習の普及を背景に、多数の AI アクセラレータが登場した。設計の共通点：

- ・ **低精度**（BF16・FP16・FP8・INT8）に最適化した**行列演算器**を大量に集積
- ・ 大容量のオンチップメモリ
- ・ 専用のデータ移動機構

一方、科学技術計算は

多くの場面で **FP64**（倍精度，仮数 53 bit）を要求する。

しかし FP64 演算器は AI 向け設計の主対象ではなく，**持たない／極端に細いチップ**が多い。

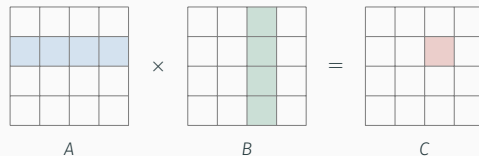
問い：低精度の行列演算器しか無い機械で，FP64 相当の行列積は構成できるか？

そして，**どこまで速く**できるのか？

なぜ行列積を対象にするのか

- ・ 科学技術計算の多くのワークロードで
計算時間の大半を占める
- ・ この種のハードウェアが最も得意とする演算
- ・ $O(N^3)$ の演算に対し $O(N^2)$ のデータ
⇒ 演算器を使い切れる形

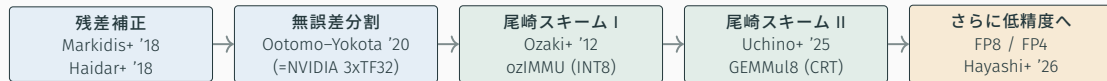
つまり、ここで届かなければ他はもっと届かない。



$$C_{ij} = \sum_k A_{ik} B_{kj}$$

「掛けて、足す」だけ

低精度から高精度を作る：これまでの流れ



- ・ いずれも **NVIDIA Tensor Core** が舞台。GPU では確立した技術になりつつある。
- ・ 他社 AI アクセラレータ（Cerebras・Graphcore・TPU）の研究は
ステンシル・FFT・PDE ソルバなどが中心で、**精度エミュレーションは手薄**。

本研究

これらを **Tenstorrent Blackhole**——Tensor Core とは設計思想の異なるチップ——へ移し、
何が成立し、何が成立しないかを実機で明らかにする。

ハードウェアとして

- ・ RISC-V ベース, Tensix コア 120 個
- ・ SIMT ではなく **明示的データフロー** の SoC 型
- ・ コアあたり L1 1.5 MB (計 ≈ 165 MB)
- ・ **FP64 演算器は無い**

開発環境として

- ・ TT-Metalium: CUDA より低位の C++ モデル
- ・ 行列エンジン・SFPU (ベクタ演算器)・NoC を **直接制御** できる
- ・ $\Rightarrow$ ハードウェアの限界を素直に測れる

研究としての位置づけ

- ・ Tenstorrent の数値計算応用は 2024 年以降の新しい分野
- ・ 既報は Grayskull / Wormhole 上のステンシル・FFT・N 体・CG
- ・ **最新の Blackhole を対象にした研究, FP32 相当を超える精度エミュレーションは見当たらない**

1. 基準性能の確定．BF16 行列積を最適化し，理論ピークの 81% (122.8 TFlops/s) を得た．
2. 精度の壁の同定．素朴な無誤差分割が実効 ≈ 13 bit で頭打ちになること，その原因が**入力の仮数長ではなく，FP 累算経路が無誤差に保持できる整数ビット幅**というハードウェア側の限界にあることを，一次資料と実測から示した．
3. 尾崎スキームの完全オンデバイス実装．スキーム I (BF16 / INT8 / FP8) とスキーム II (INT8 / FP8) の **5 経路**を，スケール算出から再構成まで**一切ホストへ戻さず**実装した．スキーム I は FP64 と triple-float (第 4 部で説明) の相互変換も**整数のビット操作だけで**デバイスに載せてある．
4. 統一比較．FP64 相当 (実効 ≈ 53 bit) で揃えた比較で，最速の Ozaki-II-INT8 が 14080^3 で 2.58 TFlops/s．
5. ハードウェアへの提言．低精度の高速モードが，誤差なしで計算するための条件のもとでは使えない機構を示した．

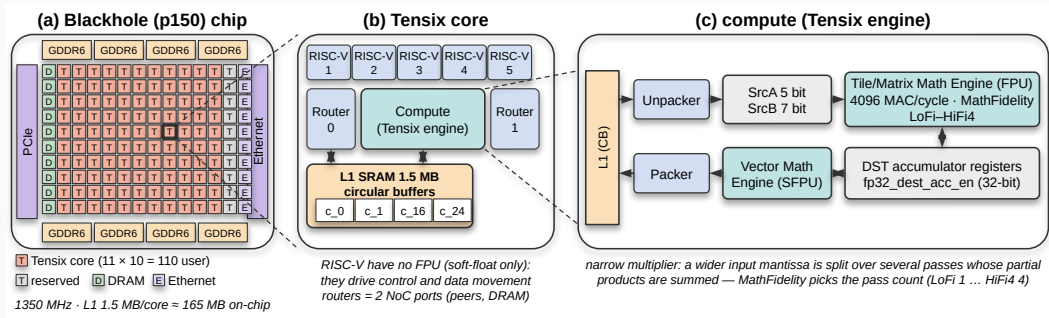
用語：本発表の「実効 n bit」

評価入力に対し，参照解（ホスト上の FP64 補償付き総和）との Frobenius ノルム相対誤差 ρ を $-\log_2 \rho$ で表したもの．**FP64 の仮数は 53 bit** なので，実効 53 bit を「FP64 相当」と呼ぶ．

これは評価入力での到達精度であって，FP64 演算との一致や正しい丸めの保証ではない． 分解・三角形の打ち切り・再構成・FP64 出力の 4 段はそれぞれ別の誤差を持つ（第 5 部）．

第1部（続）：Blackhole アーキテクチャ

Blackhole (p150) の全体像



(a) チップ：Tensix コアの格子 + NoC + DRAM (b) 1 コア：FPU / SFPU / 5 個の RISC-V / L1 (c) コンピュートエンジン内部

- ・ **行列演算器 (FPU)** : 1 サイクルに 8×16 と 16×16 の積
= $2 \cdot 8 \cdot 16 \cdot 16 = 4096$ 積和
- ・ **SFPU** : ベクタ／特殊関数ユニット
(要素ごとの演算はここ)
- ・ **5 個の RISC-V** : **FPU を持たない**
⇒ float は soft-float (≈ 76 ns/op)
- ・ **L1 1.5 MB** + 循環バッファ (CB)
- ・ **アンパッカ／パッカ** : 演算器と L1 の授受

使えるのは $11 \times 10 = 110$ コア (ディスパッチ用の予約を除く)

ここから来る制約

- ・ 要素ごとの浮動小数点計算は
SFPU が行列エンジンに載せるしかない
- ・ 宛先レジスタ DST は小さい (FP32 で 8 本)
- ・ K 全体は DST に載らないので、
部分和の**退避・読み戻し**が要る

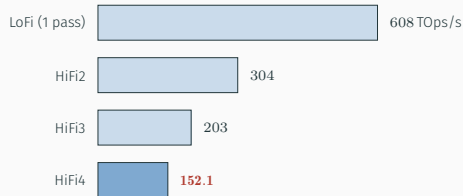
乗算器は SrcA 5 bit × SrcB 7 bit と小さい。

これより広い仮数の入力（BF16 は仮数 8 bit）は，仮数を数ビットずつ複数パスに分けて乗算・部分積加算する。

このパス数を選ぶのが MathFidelity：

- ・ LoFi（1 パス，最速）
- ・ HiFi2 / HiFi3 / HiFi4（2/3/4 パス）

スループットはパス数に反比例する。



BF16 の仮数 8 bit を丸めなく掛けるには HiFi4 が要る。
ただし尾崎が掛けるのは基数以下の小整数なので，
要るパス数は形式ではなく基数で決まる（後述）。

110 コア，1350 MHz での理論ピーク：

$$110 \times (4096/4) \times 1.35 \text{ G} = 152.1 \text{ TFlops/s}$$

パスとは何か：仮数を刻んで、部分積を足す

乗算器が 1 パスで扱えるビット幅は限られる．それより広い仮数は**刻んで**，刻みの組ごとに**部分積**を作り，パスを重ねて足す——**全部足して初めて積が厳密になる**．



模式図．内部の刻み方は非文書化なので，
ここでは 4 パス = 4 部分積として描いた．

LoFi	① のみ
HiFi2	①②
HiFi3	①②③
HiFi4	全部 ⇒ 厳密

途中で止めると「近似」にしかない

部分積を落とすので**丸めた積**になる．無誤差スキームには使えない．

実測 (INT8 radix 127)：

1.7 / 3.1 / 5.1 / 54.2 bit

「刻まなくて済む幅」に収める意味

スライスが **1 パスで収まる幅**なら，1 パス (LoFi) で厳密になる——本稿では radix 15 (FP8) と radix 31 (BF16) がこれにあたる．radix 127 (7 bit) は収まらず HiFi4 が要る (**4 倍のレート差はそもそも生じない**)．

先取り：スライスと radix（基数）

次のスライドを読むために、第3部で詳述する考え方を一言だけ先に

高精度の数を、低精度の演算器が扱える小さな整数の「桁」に割り、部分積の和として組み立て直す——これが尾崎スキームの骨格である。

- ・ 割ってできた桁を **スライス** と呼ぶ
- ・ 桁が取りうる値の上限が **radix（基数）** β
⇒ スライスは $[-\beta, \beta]$ の整数
- ・ 1 スライスが担う情報は $\log_2 \beta$ bit
⇒ β が大きいほど少ないスライスで済む＝行列積の本数が減る

$$x = \boxed{d_1} \boxed{d_2} \boxed{d_3} \boxed{d_4} \dots$$

$\longleftrightarrow$
 $\log_2 \beta \text{ bit}$

各 d_i は $[-\beta, \beta]$ の小整数 = 1 スライス
⇒ 低精度の行列演算器にそのまま載る

本発表の中心的な制約

スライスどうしを掛けて K 個足した総和が、累算器に丸めなく収まらなければならない。したがって「そのデータパスで無誤差に使える radix の最大値」が、そのままコストを決める。

入力 → 累算	無誤差 Fidelity	ピーク [TOPs/s]	無誤差整数 radix ($\approx$ bit/slice)
INT8 → INT32	HiFi4 (4)	152.1	127 (≈ 7) ; 基数によらず低 fidelity 不可
FP8(E4M3) → FP32	LoFi (1)	608	15 (≈ 4 ; SrcA 5 bit に収まる)
BF16 → FP32	LoFi (1)	608	31 (≈ 5 ; HiFi4 なら 63)
FP16 → FP32	LoFi (1)	608	31 (BF16 と同じ)

後段への布石

AI 用の**ピーク**がそのまま使えるとは限らない。

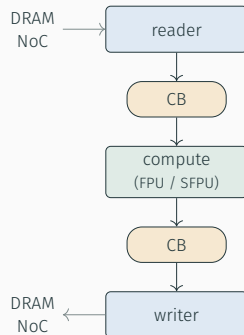
各スライスのビット幅が **LoFi の乗算器幅 (SrcA 5 bit)** に収まって初めて LoFi が使える。

INT8 の radix 127 (7 bit) は**収まらない**——この帰結は第 4 部で詳しく述べる。

GPU (SIMT + 共有メモリ階層) と異なり、
明示的データフローを設計時に記述する。

- ・ 各コア上に reader / compute / writer の 3 カーネル
- ・ 循環バッファ CB で接続
(cb_push_back / cb_wait_front)
- ・ NoC のマルチキャストでオペランド再利用
- ・ 行列エンジン・SFPU の駆動を明示制御

評価環境：Blackhole p150a 1 枚 (110 コア,
1350 MHz), TT-Metalium v0.73.1, Ubuntu 24.04.



基準：BF16 行列積をどこまで最適化できるか

実装：32 × 32 タイル単位，110 コアに出力タイル群を分配．
実行時パラメタを自動選択する．

- ・ 使用コア数 $cr \times cc$ を最大化
- ・ `in0_block_w` を L1 容量内で最大化
(= スピル往復を最小化)
- ・ サブブロック形状を整合
- ・ 出力パネル分割で任意サイズに対応

14080³ で理論ピークの 80.6%.

残り $\approx 19\%$ はスピル構造・mcast 同期・オーバーラップ不足．

サイズ N	BF16 単体 [TFlops/s (効率)]	BF16×2 [TFlops/s (効率)]
3520 ³	107.8 (70.8%)	103.1 (67.8%)
7040 ³	117.3 (77.1%)	112.0 (73.7%)
14080 ³	122.5 (80.5%)	117.0 (76.9%)

重要な構造

DST は小さく K 全体が載らない．
高占有を保つには部分和の退避・読み戻しが必要．

この経路が扱えるデータ形式を限定し，
以降の精度の壁の共通の原因になる．

第 2 部：素朴な方法と，その壁

アイデア：低精度の和で高精度を表す

BF16 は仮数 8 bit だが**指数幅は FP32 と同じ**。

したがって FP32 値 x は BF16 3 個の和へ**無誤差**に分割できる：

$$x_h = \text{RN}(x)$$

$$x_m = \text{RN}(x - x_h)$$

$$x_l = \text{RN}(x - x_h - x_m)$$

$$x = x_h + x_m + x_l$$

剰余 $x - x_h$ は FP32 で**丸めなく**求まる

(BF16 は FP32 の部分集合であるため)． x_h が上位

≈ 8 bit, x_m, x_l が続く各 ≈ 8 bit

$\Rightarrow$ 和で実効 ≈ 24 bit ($\approx$ FP32)．



それぞれが 1 個の BF16．
大きさは $2^0 : 2^{-8} : 2^{-16}$

積は「低精度の行列積の和」になる

行列 A, B を要素ごとにこう割ると ($A = A_h + A_m + A_l$, $B = B_h + B_m + B_l$)

$$AB = \sum_{\mu \in \{h, m, l\}} \sum_{\nu \in \{h, m, l\}} A_{\mu} B_{\nu}$$

すなわち 9 個の BF16×BF16 行列積の和になる。

各項の大きさは 2^{-8} 刻みで階層化されるので、
上位の項だけ採ればよい：

- $T=1$ ：実質 BF16
- $T=3$ ： $A_h B_h + A_h B_m + A_m B_h$
(= NVIDIA 3xTF32 と同型)
- $T=6$ ：積として実効 ≈ 24 bit

		B 側		
A 側		$A_h B_h$	$A_h B_m$	$A_h B_l$
		$A_m B_h$	$A_m B_m$	$A_m B_l$
		$A_l B_h$	$A_l B_m$	$A_l B_l$

緑の 3 項が $T=3$

実装上の工夫：

- ・ 出力 CB を FP32 に変更 (BF16 だと部分積が丸まる)
- ・ K ブロック間の累積をパッカーが L1 上で FP32 加算 (packer_l1_acc)
- ・ $\Rightarrow$ ホスト FP64 で総和する split- K と同値をオンデバイス完結で達成

性能は良い (2560^3 で 26.6 TFlops/s).

しかし精度が

	実効精度
BF16 単体	≈ 8 bit
BF16 $\times 2$ ($T=3$)	≈ 13 bit
$T=6$ (full)	≈ 13 bit
狙い (FP32)	24 bit

項を増やしても伸びない.

K の累積方式にも依らず一定.

$\Rightarrow$ ソフトウェアの自由度の外にある

原因の切り分け

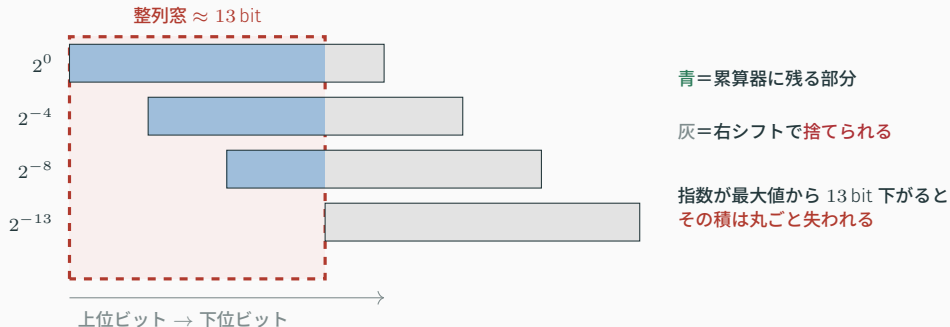
1. 単一 BF16 $\times$ BF16 積は HiFi4 でフル仮数まで無誤差
⇒ 乗算側に制約はない
2. 指数の異なる部分積の FP 総和がアラインメント丸めを受ける

公式 Tensix シミュレータで同定した機構：

1 命令内のレーン積和は、FP32 累算器の前段でグループ最大指数へ整列するブロック浮動小数点により、実効 ≈ 13 bit 粒度へ縮約される（小指数の積は右シフトで下位を捨てる）。
後段の FP32 累算（24 bit）は縮約済みの和を保持するに過ぎず、縮約前の粒度は回復できない。

一次資料も「fp32_dest_acc_en は 32-bit storage を確保するのみで 32-bit 演算を保証せず、行列エンジンの最大精度は TF32」と明記している。

図解：どこで下位ビットが落ちるか



対比

NVIDIA Tensor Core は積を完全精度で生成し真の FP32 で累積するので、同型の分割で FP32 を回復できる。Blackhole ではこの縮約が FP32 累算器の手前で起きるので、後段をいくら広くしても回復しない。

問題の本質

限界を決めているのは入力フォーマットの仮数長ではない。

FP の積・累算経路が無誤差に保持できる整数ビット幅である。

したがって：

- ・ 仮数のより長い FP16 を使っても同じ壁に当たる（同じデータパス・同じ予算）
- ・ 部分積を FP32 化する軽量改修でも消えない（+1～2 bit）

解決の方向

スライスを整数化して、累算を FP32 の無誤差整数領域 $< 2^{24}$ に閉じ込める。

整数どうしの積と和には丸め誤差が原理的に無い。

⇒ これが尾崎スキームである。

第 3 部：尾崎スキーム入門

前提

尾崎スキーム (Ozaki scheme) は既存の枠組であり、本研究の提案ではない。
本研究の貢献はこれを Blackhole の 2 つのデータパス上に実装した点にある。

以下、2 つのスキームをいちばん簡単なところから説明する：

- ・ スキーム I (Ozaki+ '12)：高精度の数を整数の「桁」に割る
⇒ 部分積が $p(p+1)/2$ 本
- ・ スキーム II (Uchino+ '25, GEMMul8)：「余り」だけを計算する
⇒ 行列積が $s \approx 16$ 本

スキーム I の直感：これは筆算である

3 桁どうしの掛け算しかできない道具で 123456×789012 を求めたい。

$$123456 = \underbrace{123}_{\text{桁}} \cdot 10^3 + \underbrace{456}_{\text{桁}}, \quad 789012 = 789 \cdot 10^3 + 12$$

展開すると

$$\underbrace{123 \cdot 789}_{\text{6 桁}} \cdot 10^6 + (\underbrace{123 \cdot 12}_{\text{6 桁}} + \underbrace{456 \cdot 789}_{\text{6 桁}}) \cdot 10^3 + \underbrace{456 \cdot 12}_{\text{6 桁}}$$

- ・ どの部分積も 6 桁なので道具に載る
- ・ 10^6 や 10^3 を掛けるのは桁をずらすだけ＝新たな誤差を生まない

高精度の数を，低精度の道具が扱える「桁」に割り，部分積の和として組み立て直す。
これが尾崎スキームの骨格である。

筆算から行列積へ：違いは 2 点だけ

1. 割るのは行列の全要素なので、上の 4 本の部分積はそれぞれ行列積 1 本になる
(1 本の行列積が全要素ぶんの部分積をまとめて計算してくれる)
2. 道具が電卓ではなく低精度の行列演算器で、そこには
「部分積とその総和を何桁まで誤差なく保持できるか」という予算がある

第 2 点が効く理由

上の筆算でも中央の項は部分積 2 本の和で、1 本より桁が要る。

行列積は内積の長さ K だけ足すので、この「足し算のぶんの桁」が本質的に効く。

$K=14080$ なら $\log_2 K \approx 13.8$ bit を、最初から予算から引かれる。

2 進で考えるときは、 10^3 が基数 β に、3 桁が $\log_2 \beta$ bit に対応する。

正規化：足す相手の「位」を揃える

浮動小数点の総和では、加数の指数がばらついていると
小さい加数の下位が**加算の丸めで落ちる**。

(これが第 2 部で見た実効 13 bit の正体)

そこで、行列積の前に行ごと・列ごとに 2 のべきで**正規化**する：

$$\tau_m^A = 2^{\lceil \log_2 \max_k |A_{mk}| \rceil}$$

- ・ 2 のべきなので**正規化そのものは無誤差**（指数シフトだけ）
- ・ 正規化後は $|A_{mk}/\tau_m^A| \leq 1$

効果が現れる点

正規化してから整数の桁に割ると、
足し合わせる相手はすべて**同じ位の整数**になる。

⇒「指数のばらつく FP 総和」に由来する
落ち方が原理的に起きない。

尾崎スキームが低精度の演算器で高精度に届くのは、
演算器の精度を上げているからではなく、
丸めが起きない領域へ問題を移しているからである。

スライス： β 進の桁に割る

基数 β を選び, $r_0 = x/\tau$ から

$$d_i = \text{round}(\beta r_{i-1}), \quad r_i = \beta r_{i-1} - d_i$$

を繰り返す (r_i が残差). 結果 $x/\tau = \sum_i d_i \beta^{-i}$.

数値例: $\beta=4$, $x=41/64$

$$r_0 = 41/64 = 0.640625$$

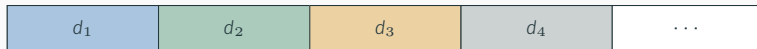
$$4r_0 = 2.5625 \Rightarrow d_1 = 3, \quad r_1 = -0.4375$$

$$4r_1 = -1.75 \Rightarrow d_2 = -2, \quad r_2 = 0.25$$

$$4r_2 = 1 \Rightarrow d_3 = 1, \quad r_3 = 0$$

$$\frac{41}{64} = 3 \cdot 4^{-1} - 2 \cdot 4^{-2} + 1 \cdot 4^{-3}$$

- 丸めが最近接なので桁は符号つきになる (平衡表現)
- 第 2 桁以降は $|d_i| \leq (\beta+1)/2$. 先頭桁は正規化しだいでこれを超えうる (左の例の $d_1=3$) ので, 予算は保守的に $|d_i| \leq \beta$ で見積もる
- 1 スライスが担うのは $\log_2 \beta$ bit
 $\Rightarrow \beta$ を大きく取るほどスライス数 p が減る



x/τ の β 進展開

$\log_2 \beta$

部分積と、三角形の打ち切り

A, B を p スライスに割ると

$$C_{mn} = \sum_{i,j} \tau_m^A \tau_n^B \beta^{-(i+j+2)} P_{mn}^{(i,j)}$$

$$P_{mn}^{(i,j)} = \sum_k a_{mk}^{(i)} b_{kn}^{(j)} \quad (\text{整数 GEMM})$$

全対は p^2 個。しかし $i+j \geq p$ の対（右下の三角）を捨てて

生じる誤差は相対 $\approx \beta^{-(p+1)}$ で、 p が定める到達精度と同水準。
 $\Rightarrow i+j \leq L$ （本発表は $L=p-1$ ）を採り、本数は

$$p(p+1)/2$$

($d=0$ が 1 個, $d=1$ が 2 個, $\dots$, 計 $1+2+\dots+p$)

p を 1 増やすとコストは $p+1$ 本増える。

コストが p の 2 乗で効くのは、この三角形の形による。

		j (Bのスライス)					
i (Aのスライス)	0	1	2	3	4	5	
	1	2	3	4	5		
	2	3	4	5			
	3	4	5				
	4	5					
	5						

数字は対角 $d=i+j$
同じ対角は同じ重み $\beta^{-(d+2)}$
灰色は捨てる ($p=6$ なら 15 個)

桁の絶対値を保守的に β 以下と見ると部分積 1 つは β^2 未満. これを K_{eff} 個足すと $K_{\text{eff}}\beta^2$ 未満.

無誤差条件

$$2w + \lceil \log_2 K_{\text{eff}} \rceil \leq b_{\text{acc}} \quad (\beta = 2^w - 1)$$

b_{acc} : 累算器が丸めずに保持できる整数ビット幅

経路	b_{acc}	$K=14080$ で
INT8 $\rightarrow$ INT32	31	$\log_2 \beta \leq 8.6$
BF16/FP8 $\rightarrow$ FP32	24	$\log_2 \beta \leq 5.1$

演算器が速いことと、無誤差でいられる幅が広いことは別である.

尾崎スキームのコストは、この予算がどれだけ取れるかでほぼ決まる.

b_{acc} はフォーマットの仮数長ではなくデータパスの実装で決まるため、実機で同定する必要がある.

スキーム 1: まとめ

1. 行／列ごとに 2 べきで**正規化**
2. 基数 β で**整数スライス**に分解
3. 対 (i, j) ごとに**整数 GEMM** (無誤差)
4. 重み $\beta^{-(d+2)}$ で**再構成**

到達精度

スライス数 p に対し**飽和せず**伸びる.

⇒ **必要な精度を p で選べる**

コスト

$\frac{p(p+1)}{2}$ 本の GEMM		
経路	β	FP64 相当に要る p
BF16	31	11 (66 本)
INT8	127	8 (36 本)
FP8	15	14 (105 本)

実測 ($\beta=31, 1280^3$):

$p=8/9/10/11/12 \Rightarrow 39.7/44.6/49.5/53.8/54.2$ bit

スキーム I の弱点

FP64 相当には $p(p+1)/2$ 本の GEMM が要る (INT8 で 36 本, BF16 で 66 本).
この分解と目標精度のもとでは**不可避**——最適化できるのは定数項だけ.

別の道

行列積の本数そのものを減らす方式がある.
それが尾崎スキーム II (Uchino+ '25, 実装名 GEMMul8).
FP64 相当を $s \approx 16$ 本で得る (36 本に対し $\approx 2.25\times$ 少).

道具は**整数の「余り」**と**中国剰余定理**. 順に準備する.

準備 1: 余りは足し算と掛け算を保つ

合同式の基本性質

$a \equiv a', b \equiv b' \pmod{m}$ ならば

$$a + b \equiv a' + b', \quad ab \equiv a'b' \pmod{m}$$

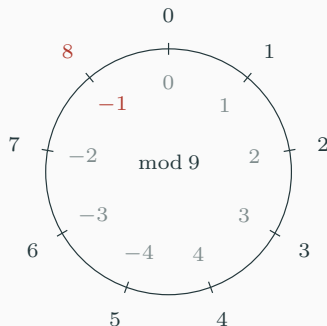
行列積の中身は「掛けて足す」だけである。したがって

$$C' \equiv (A' \bmod m) (B' \bmod m) \pmod{m}$$

- ・ 右辺は各要素が $\pm m/2$ 以内の小さな整数どうしの、ごく普通の行列積
- ・ $m \leq 255$ に選べば要素は $\pm 127 \Rightarrow \text{INT8}$ 行列積エンジンがそのまま使える
- ・ 掛けて足した総和も 32 bit 整数に無誤差で入る

「16 桁の精度が欲しい」が「8 bit 整数の行列積を s 本」に化ける。

準備 2：余りの選び方の自由（対称代表元）



9 離れた数 ($\dots, -10, -1, 8, 17, \dots$) は**同じマス**.
どの代表を書くかは自由：

- ・ 外側 $\{0, \dots, 8\}$ ：学校で習う流儀
- ・ 内側 $\{-4, \dots, 4\}$ ：**対称代表元**

対称に選ぶ理由

絶対値が最大 $m-1$ ではなく $m/2$ で済む.
余りどうしを掛けて足すので、積の大きさは
 $(m-1)^2$ 対 $(m/2)^2$ で**およそ 4 倍**違う.

「総和が 32 bit からあふれないうちは無誤差」
という制約下で、この 4 倍は大きい.

準備 3：中国剰余定理

余り 1 個では元の数を決まらない
(「7 で割ると 2 余る数」は 2, 9, 16, ...).

しかし互いに素な法を複数使えば決まる.

例: $m=3, 5, 7$ ($M=105$) で

$$23 \equiv (2, 3, 2)$$

0~104 でこの組を与えるのは 23 だけ 1 つ.

⇒ 幅 M の「窓」の中でなら、
余りの組が元の数に完全に特定する.

		mod 5 の余り				
		0	1	2	3	4
mod 3 の余り	0	0	6	12	3	9
	1	10	1	7	13	4
	2	5	11	2	8	14

$m=3, 5$ ($M=15$): 15 マスに重複なく 1 回ずつ.
組 (1, 3) なら 13 (赤).

2 つの注意

(1) 窓の外は区別できない ⇒ M は答えの取り得る範囲を覆うだけ大きく. (2) 法は互いに素でなければならない.

準備 4：逆元と復元の公式

逆元 = mod の世界での**逆数**.

a と m が互いに素なら $ay \equiv 1 \pmod{m}$ を満たす y が一意に存在.

例: mod 13 で $8 \cdot 5 = 40 = 3 \cdot 13 + 1$ より $8^{-1} \equiv 5$.

$\Rightarrow$ 「5 を掛ける」が「8 で割る」の代わり.

(法が決まった時点で一度だけ前計算すればよい量)

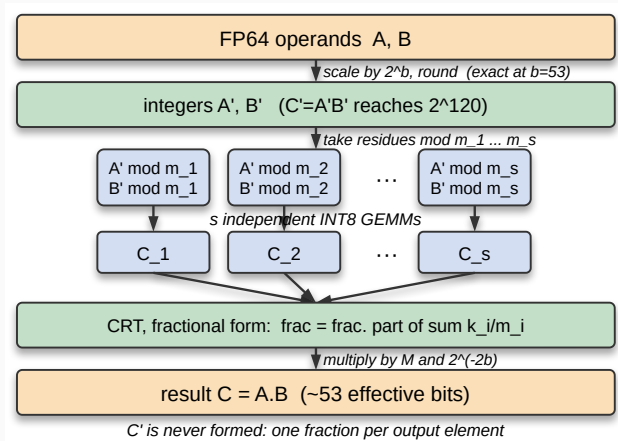
復元の公式

$$x \equiv \sum_{i=1}^s c_i \mathcal{M}_i y_i \pmod{\mathcal{M}}$$

$$\mathcal{M}_i = \mathcal{M} / m_i, \quad y_i = \mathcal{M}_i^{-1} \pmod{m_i}$$

なぜこの形か: $\mathcal{M}_i$ は m_j ($j \neq i$) で割り切れるので、そこでは余り 0. mod m_i では中途半端な値だが、逆元 y_i を掛けて余り 1 に**矯正**される.

$\Rightarrow \mathcal{M}_i y_i$ は「 i 番目の法でだけ 1, 他では 0」という**スイッチ**.



巨大な整数積 C' は一度も作らない——法ごとの余りだけを s 本の INT8 行列積で求める。

第1段：小数を整数の問題に直す

$$A' = \text{round}(A \cdot 2^b), \quad B' = \text{round}(B \cdot 2^b)$$

求めたい積は**整数の行列積** $C' = A'B'$ になり、 $C = C' \cdot 2^{-2b}$ で戻せる。

(A 側と B 側の**両方**を 2^b 倍したので、戻しの指数は b でなく $2b$)

整数どうしの掛け算と足し算なので、 C' には**原理的に丸め誤差がない**。

問題： C' が巨大

$$|C'| \leq K \cdot 2^{2b}$$

$b=54$ (FP64 を丸めなく捕捉), $K=14080$ なら 2^{122} 。

演算器は 32 / 64 bit 単位なので直接は扱えず、ソフト多倍長は極端に遅い。

ここで「余り」の出番。

第2段 → 第3段：分数形での復元

第2段： C' そのものは求めず， s 個の法での余りだけを求める．

$$C' \equiv (A' \bmod m)(B' \bmod m) \pmod{m}$$

- ・ 法どうしは完全に独立 $\Rightarrow s$ 本を並列に
- ・ 各行列積は普通の INT8 GEMM

第3段：復元公式をそのまま使うと x が M に迫る巨大整数になる．

$\Rightarrow$ 両辺を M で割る：

$$\frac{C'}{M} \equiv \sum_{i=1}^s \frac{k_i}{m_i} \pmod{1}$$

$k_i = (c_i y_i) \bmod m_i$ ．右辺は1未満の分数を s 個足すだけで，どの中間値も s 程度．

小数部 frac が求まれば $C = \text{frac} \cdot M \cdot 2^{-2b}$ ．

なぜ小数部だけでよいか

窓を $M > 2 \max |C'|$ に取れば C'/M は初めから $(-1/2, 1/2]$ に収まっているので，畳んだ小数部は C'/M そのもの． $\Rightarrow$ 巨大な C' を一度も作らずに済む．

通しの数値例（手で追える大きさで）

長さ $K=2$ の内積 $u = (0.75, -0.5)$, $v = (0.5, 0.25)$, $u \cdot v = 0.25$ を幅 $b=4$ で解く.

第 1 段: $U=(12, -8)$, $V=(8, 4)$. $C'=12 \cdot 8 + (-8) \cdot 4 = 64$, $C=64 \cdot 2^{-8}=0.25$.

窓: $|C'| \leq K2^{2b}=512$ より $M > 1024$ が要る. $m=(13, 11, 9)$ で $M=1287$.

第 2 段: 法ごとの余り（対称代表元）と内積

第 3 段: $\mathcal{M}_i=(99, 117, 143)$, 逆元 $y_i=(5, 8, 8)$,
 $k_i=(c_i y_i) \bmod m_i = (-5, -5, -1)$

法 m	$U \bmod m$	$V \bmod m$	内積	$C' \bmod m$
13	$(-1, 5)$	$(-5, 4)$	$5 + 20 = 25$	-1
11	$(1, 3)$	$(-3, 4)$	$-3 + 12 = 9$	-2
9	$(3, 1)$	$(-1, 4)$	$-3 + 4 = 1$	1

$$\sum_i \frac{k_i}{m_i} = -\frac{5}{13} - \frac{5}{11} - \frac{1}{9} = -\frac{1223}{1287}$$

小数部を $(-1/2, 1/2]$ へ畳む (+1 するだけ):

$$\text{frac} = \frac{64}{1287}$$

$$C' = \text{frac} \cdot M = 64$$

$$C = 64 \cdot 2^{-8} = 0.25 \checkmark$$

この 3 行が、実機では 3 本の独立な INT8 行列積.

行列積エンジンが出すのは「内積」列（畳む前の生の総和）だけ.

2つのスキームの比較

	スキーム I	スキーム II
分解の道具	β 進の桁 (スライス)	互いに素な法での余り
GEMM 本数	$p(p+1)/2$ (INT8 で 36)	s (≈ 16)
GEMM 間の依存	対角ごとに重み付き総和	完全に独立
再構成	Horner ($\times \beta$ と加算. 第 4 部)	CRT, 除算・剰余を含む
再構成の重さ	≈ 54 演算/要素と軽い	重い
精度のつまみ	スライス数 p	整数化ビット幅 b (2^b 倍して整数化)

本研究の要点

2つのスキームはハードウェアへの要求が対照的.

スキーム II は整数演算器が行列エンジンに対し強い GPU で有利.

スキーム I は SFPU が相対的に弱い, 低精度演算器中核のアクセラレータに向く.

⇒ Blackhole ではどちらが勝つか? (第 5 部)

第 4 部：Blackhole への実装

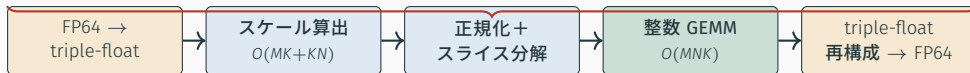
なぜ厳守するのか

$O(MK)$ / $O(MN)$ の中間データをホストへ戻せば、PCIe とホストの単スレッドが実時間を支配してしまい、アクセラレータの評価にならない。

本研究の目的はこのデバイスに実装する際の問題を洗い出すことなので、性能が落ちてもオンデバイスで実装し、その考察をする。

計時対象（DRAM 常駐後）の全工程をデバイス上で実行する：

計時区間：すべてデバイス上



scheme I ではホストに演算が一切残らない——FP64 をタイル順に並べて渡し、FP64 のまま並べ戻すだけである。両端の変換はデバイスの RISC-V が整数のビット操作だけで行う（double で書くと soft-float になって 13-17 倍かかる）。

そもそも FP64 はどこにも無い：三つの演算資源

コア内の演算資源は三つで、**どれも FP64 を持たない**。何をどこに載せるかが実装のすべてを決める。

資源	できること	高精度計算での役割
行列エンジン	BF16 / INT8 / FP8 の積和, FP32 または INT32 で累算	スライス対の GEMM
SFPU (ベクタ)	FP32 のタイル加減算・乗算. FMA も move も無い	スライス分解と Horner 再構成
RISC-V (5 個/コア)	整数演算のみ. FPU 無し	DRAM ↔ L1 の移動と, FP64 ↔ triple-float 変換

「FP64 で足せばいい」が使えない

- SFPU は FP32 まで. 53 bit を保つには**複数語**にするしかない ⇒ triple-float
- RISC-V で double を書くと libgcc の soft-float (≈ 76 ns/op). 実測で **13-17 倍** ⇒ 整数ビット操作
- ハード FP64 を持つ big RISC-V (SiFive X280) は TT-Metalium のカーネル対象外

整数で書くと、速いだけでなく正確

2 のべきでない最終スケールを FP64 の定数で掛けると、そこで**相対 2^{-53} が全要素に一様に乗る**.
64 bit 仮数との整数乗算にすれば 2^{-64} .
⇒ scheme I-INT8 53.0 → 54.2 bit

まず全体像：スライスの積は「筆算」である

第3部で出した**対角**を，実装の側から見直しておく．スライス分解は x/τ を β 進の**小数**に書くことだった：

$$x/\tau = a_1\beta^{-1} + a_2\beta^{-2} + \dots$$

2つを掛けると， β 進の**小数**どうしの**筆算**になる：

$$a_i b_j \text{ の重みは } \beta^{-(i+j+2)}$$

重みは $i+j$ だけで決まるので， $i+j$ が同じマスは同じ桁に落ちる．

⇒ 筆算の「たて1列」にあたるのが**対角**
 $d=i+j$ ．

GEMM 段がやること

対角 1 本につき整数 GEMM 1 本で，**その列の合計 P_d** を出す（無誤差）．

	b_1	b_2	b_3	b_4	数字=その積が落ちる桁 d
a_1	0	1	2	3	
a_2	1	2	3	4	$d=2$ の対角
a_3	2	3	4	5	
a_4	3	4	5	6	

白いマスは $i+j > L$ ．
小さすぎるので捨てる

同じ色=同じ桁で，色の数だけ GEMM を走らせる ($L+1$ 本)．残す対は $p(p+1)/2$ 個．

再構成＝桁から数を組み立てる (Horner)

列ごとの合計 P_d が揃った．あとは桁を数に戻すだけである：

$$C = \sum_d \beta^{-(d+2)} P_d = \underbrace{\left(\sum_d \beta^{L-d} P_d \right)}_S \cdot \beta^{-(L+2)}$$

十進の位取りと同じ手順

$\beta=10$ ，桁が 3, 1, 4 のとき $S=314$ ．

上の桁から「10 倍して次を足す」：

$$3 \rightarrow 3 \cdot 10 + 1 = 31 \rightarrow 31 \cdot 10 + 4 = 314$$

これが Horner．位取りの読み方そのものである．

ここでも同じ

running sum を β 倍して P_d を足す，を $L+1$ 回：

$$S \leftarrow S \cdot \beta + P_d$$

最後に $\beta^{-(L+2)}$ を一度だけ掛ける（出口の整数乗算）．

なぜ β^{L-d} を直に作らないのか

$S = \sum_d \beta^{L-d} P_d$ は、重み β^{L-d} を定数として作って掛けても計算できる。しかしそれは損である。

定数を作る場合

β は非 2 べき (31/127/15) なので、 β^{L-d} は丸めた定数にしかない。

$\Rightarrow L+1$ 個の重みがそれぞれ別の誤差を持ち込む。

Horner の場合

掛けるのは毎回 β そのものだけ。

しかも β が小さいので、その積は無誤差にできる (次頁)。

$\Rightarrow$ 重みに由来する誤差が一つも入らない。

加えて、 P_d は GEMM 段から対角の順に 1 つずつ出てくる。Horner はその順に消費するので、 $L+1$ 本を溜めずに流せる。

課題 1: この累積を, FP64 の無い機械でどう保つか

Horner の running sum は $L+1$ 段 (8-14 段) にわたって β 倍と加算を受け続ける.
これを FP32 単独で回すと累積が ≈ 24 bit で飽和し, FP64 入力の 53 bit を保てない.

triple-float

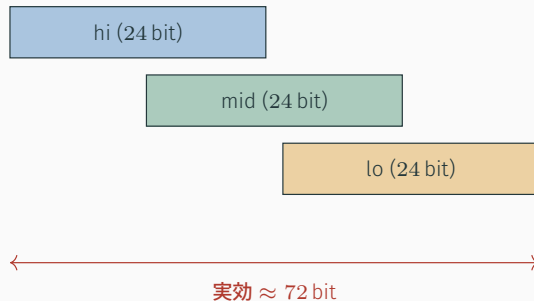
値を 3 本の FP32 の非評価和で保持:

$$x = \text{hi} + \text{mid} + \text{lo}$$

(3 本のそれぞれを limb と呼ぶ)

$$|\text{mid}| \leq \frac{1}{2} \text{ulp}(\text{hi}), |\text{lo}| \leq \frac{1}{2} \text{ulp}(\text{mid})$$

仮数を約 3 倍 = 実効 ≈ 72 bit に拡張.



FP64 上の triple-double / triple-word の FP32 版.

必要な演算は 2 つだけ

triple-float + FP32 と triple-float $\times$ 小整数基数 β . 後者は $\beta \leq 127$ が小さく Dekker 分割で積が無誤差になるので, 一般の乗算より短い手順で済む.

基礎は各 FP32 演算の丸め誤差を，追加の FP32 として回収すること．

TwoSum

$$t = \text{fl}(a + b), \quad e = (a + b) - t$$

e は丸めた分ちょうどを表す FP32.

TwoProd (Dekker)

$$u = \text{fl}(ab), \quad e = ab - u$$

ハードウェアの制約

本ハードウェアの SFPU はタイル単位の加減算・乗算のみで FMA を持たない．

⇒ TwoProd は $e = \text{fma}(a, b, -u)$ の 1 演算版ではなく，

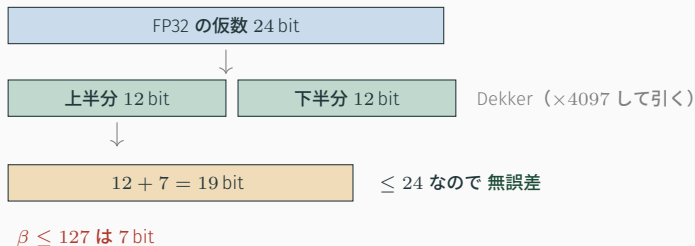
Veltkamp 分割版を用いる（手順が長い）．

もう一つの罫：TT-Metalium は既定で `-ffast-math` 付きで JIT ビルドする．無誤差変換を C の float 式で書くと残差の式が代数的に 0 へ簡約され，誤りを出さないまま精度が FP32 相当へ落ちる（本実装は SFPU 組込み関数なので影響なし）．

残差を下位成分へ順に伝播させることで，個々の FP32 演算が 24 bit に丸めても 3 本組は ≈ 72 bit を保つ．

なぜ「 $\times \beta$ 」だけは無誤差にできるのか

FP32 の仮数は 24 bit. 一般の掛け算は積が 24 bit を超えて下位が落ちる.
ところが β が小さいおかげで、落とさずに済む.



手順

1. 各 limb を上下 12 bit に割る
2. それぞれに β を掛ける (どちらも無誤差)
3. 足し戻し, こぼれを残差として拾う

一般の乗算だったら

残差を回収する TwoProd が要る. しかも本ハードウェアの SFPU には FMA が無いので Dekker 版を使うことになり, 手数がさらに増える.

$\Rightarrow \beta$ が小さいことが, そのまま手数の削減.

同じ $\times\beta$ が、スライス段でも使われる

$\times\beta$ が必要なのは再構成だけではない。スライス分解も、 β 進の桁を上から取り出すために同じ演算を使う (T は残差の 3 本組)：

$$d = \text{round}(T), \quad T \leftarrow (T - d) \cdot \beta$$

掛けたあと、必ず 3 本組へ戻す

$\times\beta$ の結果は 4 成分に散らばるので、TwoSum の鎖で非重複な 3 本組に整えて返す。

省けない理由：桁 d を引いたあとの上位 limb は小さくなっている、中位 limb が相対的に大きくなる。整えておかないと、次の $\text{round}(T)$ が上位 limb だけを見て誤った桁を返す。

scheme II では基数が 2^{20}

2 のべきなので、limb ごとに指数をずらすだけ。
 $\Rightarrow$ Dekker 分割も残差の回収もすべて不要になり、残るのは正規化の鎖だけ。

それでも省けないのは、桁を取り出す理由が同じであるためである。

つまり $\times\beta$ と $+FP32$ の 2 演算が、スライス段と再構成段の両方を賄っている。

残るは両端：FP64 と triple-float をどう行き来するか

デバイス内は triple-float で回せるようになった．あとは**入口**で FP64 から triple-float へ，**出口**で triple-float から FP64 へ移すだけである——それも**浮動小数点演算を使わずに**．

入口 (FP64 → triple-float)

- ・ リーダが 8 バイト要素のまま読み，**指数フィールドと仮数のビット操作**だけで 3 本の limb に割る
- ・ 正規化 (2 のべき) は FP64 の**指数への整数加算**として同じパスに畳む

出口 (triple-float → FP64)

- ・ ライタが 3 本を整数のアクキュレータへ集めて組み立てる (幅は scheme I が 64 bit, **scheme II は 128 bit**)
- ・ $\tau_m^A \tau_n^B$ は**指数へ足すだけ**．2 のべきでない最終スケールだけ 64 bit **仮数との整数乗算**で掛ける

なぜ整数で書くのか

データ移動を担う RISC-V は FPU を持たない．double で素直に書くと soft-float になり， 4096^3 の実測で**入口** 12.9× / **出口** 16.8×．

副産物：精度が上がる

最終スケールを FP64 の定数で掛けると**相対 2^{-53} が全要素に一様に乗る**．64 bit 仮数の整数乗算なら 2^{-64} ．

scheme I-INT8 53.0 → 54.2 bit．

入口の中身：53 bit を 23/23/7 に割り，CLZ を避ける

FP64 の 2 語 (w_1, w_0) から，浮動小数点演算を一切使わずに 3 本の limb を作る．

1. 仮数を三つに割る．上から a (23) / b (23) / c (7) へ．境目は最近接に丸め， b は符号付きにするので片寄りが出ない．
2. 指数を作る． $P = E - 1075 + d$ ． d が正規化で，指数フィールドへの整数加算に畳まれている．
3. 上位は FP32 として直接組む．指数 $P+179$ ，仮数 a をそのまま置く．
4. 下位 2 本は組まない． b, c から FP32 を直に作るには先頭ビット位置の探索 (CLZ) が要る．

CLZ を避ける「マジックナンバ」

正規化済みの値に足し込んだ形で出し，正規化は SFPU の定数減算に任せる：

$$\text{braw} = 2^{23} + 2^{22} + b$$

$$\text{craw} = 2^{23} + c$$

$$s = \pm 2^{(P+7)}$$

s が中位の位 2^{P+7} なのは， 2^P だと s だけが早く落ちて中位・下位が同時に消えるためである．リーダは整数の or と加算だけ．SFPU 側は

$$\text{mid} = (\text{braw} - 12582912) \cdot s$$

$$\text{lo} = (\text{craw} - 8388608) \cdot s \cdot 2^{-7}$$

の 5 演算で済む．

出口の中身：整数アキュムレータに集めて FP64 を組む

1. 3 本を整数へ．各 limb を（符号付き仮数，指数）に分解し，**最大の指数**を基準にして整数アキュムレータへ足し込む．
2. 寄せ幅は 37 bit．再構成の出力は**正準な 3 本組とは限らず**，中位が上位と同じ位に来ることがある．39 で寄せると 3 本の和が **int64 を越えて符号が反転した** (7040^3 の FP8 で 1 要素)．37 なら各項が 2^{61} 未満で安全．
3. 窓の下へ落ちきった limb は記録しておく．上位が相殺したときに基準を取り直して集め直すため（0 を返さないため）である．
4. 最終スケールを整数乗算で．64 bit へ正規化してから W （64 bit 仮数）と **32 bit 乗算 4 回**で 128 bit 積を作る（RV32 に 64 bit 乗算は無い）．
5. 丸めて指数を組む．出力は真値から 1 ulp 以内で，同点の向きは規定しない（方式はこれに依存しない）．

ホスト側の模擬（tools/tfconv）が `__float128` 参照と突き合わせて全手続きを検査する．

なぜ上位語の先頭ビットが 2 択か

A も W も $[2^{63}, 2^{64})$ なので積は $[2^{126}, 2^{128})$ ．
上位語の先頭は**ビット 62 か 63**にしか来ず，
落とす幅は 10 か 11 の 2 択に固定できる．

⇒ RV32 で重い**可変 64 bit シフトの展開が消える**．

保証すること／しないこと

分割は**厳密**（hi+mid+lo が入力に一致）．
組み立ては**真値から 1 ulp 以内**．**同点の向きは規定しない**．

最近接偶数を実装しないのは，方式がそこに依存しないためである（桁の上限は clamp が構成的に与え，CRT の窓は狭義，ModRound の同点は両向きとも同じ剰余類）．

課題 2：無誤差条件と K 方向のブロック化

本実装は同一対角のスライス対を K 方向に連結して 1 本の GEMM にする

⇒ 中間読み出しが $O(p^2) \rightarrow O(L)$ に減る

⇒ しかし $K_{\text{eff}} = (L+1)K$ となり条件が $(L+1)$ 倍きびしい

対処： K をブロックに分ける

K を長さ K_c に分け、ブロックごとに尾崎を回して triple-float で足し込む。

Horner は P_d について線形なのでそのまま畳める。

⇒ 条件が K でなく K_c にかかり、
基数もスライス数も K に依存しなくなる。

なぜ実測に頼れないか

乱数入力では部分和が符号の相殺で乱歩となり、
条件を破っても見かけ上は正しい値が出る。

非負行列で確かめると、BF16 経路は $\beta=63$,
 $K=7040$ で

$53.2 \rightarrow 17.4 \text{ bit}$ へ崩れる。

⇒ 構成時に条件を**検査**する。

基数はハードウェア上限そのものではなく**ブロック長との釣り合い**で選ぶ：

BF16 経路 7040^3 の実測で

$\beta=63/31/15 \Rightarrow 0.44/0.57/0.43 \text{ TFlops/s}$ 。

課題 3：スライス数 p を誰が決めるのか

到達精度は p とともに飽和せず伸び、triple-float 再構成の上限 (≈ 53 bit) で止まる．

$\beta=31$ ，立方 1280^3 の実測

p	8	9	10	11	12
到達精度 [bit]	39.7	44.6	49.5	53.8	54.2

p は「最小で足りる値」を選ぶ

p を 1 増やすと GEMM が $p+1$ 本増える．

$p=11 \rightarrow 12$ なら $66 \rightarrow 78$ 本 = 18% のコスト増で，得られる精度は 0.1 bit.

つまり「ここで止めてよい」と判断する規則（停止則）が要る．そして本ハードウェアでは，古典尾崎の停止則がそのままでは使えない．

基数を 2 べき $\beta=2^w$ に取ると，1 スライスがちょうど w ビットを抜き出す．

2 進小数は有限桁なので，有限回で抜き尽くせる．

⇒ 残差が厳密に 0 になったら止めればよい．

- ・ 判定はただの比較——誤差解析も参照解も要らない
- ・ 入力依存に自動決定できる（易しい入力なら早く止まる）
- ・ GPU の先行実装（ozIMMU，OzBLAS）はこれを使う

$\beta=4$ ， $x=41/64$ （第 3 部の例）

βr_{i-1}	d_i	残差 r_i
+2.5625	+3	-0.4375
-1.75	-2	+0.25
+1.0	+1	0

3 桁で残差が 0 に落ちる ⇒ ここで停止

ところが、2 べき基数が使えない（理由 1）

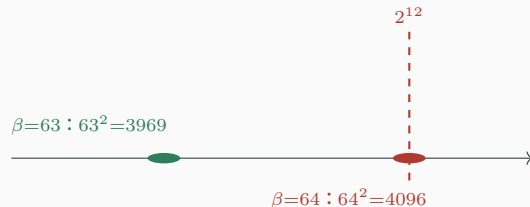
$\beta=64$ にすると精度が崩壊する

立方 1280^3 , $p=8$ の実測

基数	到達精度
$\beta=63$	47.9 bit
$\beta=64$	14.0 bit

1 増やしただけで、第 2 部で見た **13 bit の壁** と同じ水準まで落ちる。

理由：スライス積がちょうど 2^{12} に届く。



2^{12} に届くか、未満に収まるかの差である。届いた瞬間にグループ最大指数が 1 つ上がり、**整列窓が 1 指数分ずれて最下位が落ちる**（第 2 部）。

2 べきに固執するとコストが跳ねる (理由 2)

無誤差条件 $2w + \lceil \log_2 K_{\text{eff}} \rceil \leq b_{\text{acc}}$ が効くのは w (=桁を収めるビット幅) である。
したがって条件は、 w ビットに収まる**最大**の値 $\beta=2^w-1$ を選ぶことに報いる。

基数 β	要る幅 w	bit/slice = $\log_2 \beta$	予算の消費 $2w$	K ブロック長 K_c
31	5	4.95	10	基準
32	6	5.00	12	1/4
63	6	5.98	12	基準
64	7	6.00	14	1/4 (かつ理由 1 で崩壊)

$\beta=32$ は $32 > 2^5-1$ なので $w=6$ が要り、 $2w$ が 2 ビット増える。

⇒ K_c が 1/4 に縮み、再構成の回数が 4 倍になる。

一方で得られる bit/slice は $4.95 \rightarrow 5.00$ とほとんど変わらない。

非 2 べきでは、残差が 0 に落ちない

同じ $x=41/64$ を $\beta=31$ で割ると

βr_{i-1}	d_i	残差 r_i
+19.859375	+20	-0.140625
-4.359375	-4	-0.359375
-11.140625	-11	-0.140625
-4.359375	-4	-0.359375
-11.140625	-11	-0.140625
⋮ 以下、永久に循環		

- ・ 残差は周期 2 の循環に入り、二度と 0 にならない
- ・ 桁も $-4, -11, -4, -11, \dots$ と $O(\beta/2)$ のまま出続ける
- ・ $\beta=4$ なら 3 桁で終わった同じ入力である

なぜか——十進で $1/3$ が循環するのと同じ

β 進展開が有限桁で終わるのは、値が β のべきの分数で書けるときだけ。

2 進小数 (= 分母が 2 のべき) を $\beta=31$ 進で書こうとすると、31 と 2 が互いに素なので割り切れない。

⇒ 残差 0 終了は原理的に発火しない。

解：誤差限界そのものを停止則にする

残差が 0 になるのを待てないなら、「あと何ビット残っているか」を見積もって止める。

Abdelfattah らのデータ依存誤差限界

p 桁で打ち切ったときのスライス誤差は

$$\approx \beta^{-p} \kappa |A||B|$$

κ ：入力の各行・各列の動的域 max / mean から得る係数

⇒ $O(MK+KN)$ で安価に計算でき、高精度参照計算を要さない

$\beta^{-p} \kappa \lesssim 2^{-51}$ を満たす最小の p を選ぶ

- ・ 閾値 2^{-51} は triple-float 再構成の床 (≈ 53 bit) に対応する
- ・ 限界は約 2 bit 保守的なので、実測上限に一致する値を用いる

経路	β	返す p	GEMM 本数
BF16	31	11	66
INT8	127	8	36
FP8	15	14	105

誤差限界自体は既存手法である。本研究の寄与は、これを非 2 べき基数の停止則として本ハードウェアに適用した点にある。

3つのデータパスへの具体化（スキーム I）

Ozaki-I-BF16 ($\beta=31$)

- ・ **積バジェット**（累算器が丸めずに保持できる整数積の上限）を実機で掃引
 $\Rightarrow w \leq 6$ で無誤差, $w \geq 7$ で **破綻**という鋭い境界
- ・ BF16 は ≤ 256 の整数を無誤差表現するので、**スライス**をそのまま**出力**できる
- ・ $p=11$ （66 本）

Ozaki-I-INT8 ($\beta=127$)

- ・ INT32 累積は整数加算ゆえ無誤差
- ・ K スピルの 2 経路のうち **c_24 経路は無誤差**, **パッカ L1 累積は FP32 固定で int32 下位を失う**
- ・ $b_{acc}=31$ と広く、全サイズ・ $p \leq 9$ で **K ブロック化が不要**
- ・ $p=8$ （36 本）

Ozaki-I-FP8 ($\beta=15$)

- ・ radix が小さく **LoFi に乗れる**（608 TOps/s）
- ・ 枠組はフォーマット非依存なので**マクロ再ビルドだけ**
- ・ radix 15（4 bit）は SrcA 5 bit に収まるので **fidelity に依らず無誤差**
- ・ $p=14$ （105 本）

FP16 を使わない理由

radix 上限を決めるのは **FP 積・累算経路が無誤差に保持できる整数ビット幅**であって入力の仮数長ではない。

BF16 は 127 を無誤差表現できるのに $w \geq 7$ で破綻する——ゆえに仮数のより長い FP16 でも**同じ壁**。

INT8 の落とし穴：4 倍レートが使えない

素朴には、INT8 の LoFi 1 パス (608 TOPs/s) が HiFi4 (152) の 4 倍速で有利に見える。

fidelity 掃引の実測 (1280^3 , $p=8$)

Fidelity	実効精度
LoFi	1.7 bit
HiFi2	3.1 bit
HiFi3	5.1 bit
HiFi4	54.2 bit

理由は基数の幅ではない

radix 127 は 7 bit で SrcA 5 bit を超える——が、radix 31 (5 bit) に落としても同じ：

HiFi4 53.8 / HiFi3 1.0 / HiFi2 1.0 / LoFi 0 bit

⇒ どの基数でも INT8 経路は低 fidelity で答えが出ない。

符号-絶対値 UInt8 の符号が最上位にあることが疑われるが、

一次資料に記述がなく機構は同定できていない。

LoFi に乗らないのは 5 経路のうち INT8 だけである。期待した 4 倍のレート差は、そもそも実現しない。それでも INT8 が BF16 を上回るのは、1 バイトのオペランドでデータ供給が有利で占有率が高いこと、そして部分積数が 36 対 66 と少ないことによる。

再構成は GEMM の生の総和 C_i を法 m_i で剰余してから CRT する.

これを FP32 SFPU で行うので

$$|C_i| = K r_{\max}^2 < 2^{24}$$

が要る.

この上限をそのまま課すと法は K とともに痩せる：
 $K=14080$ では $m_i \leq 69$ となり，法テーブルの最小値
173 すら使えず法が 1 本も取れない.

対処：上位・下位に割ってから法をとる

$$|C_i| = c^{\text{hi}} 2^{16} + c^{\text{lo}}$$

c^{hi} は 15 bit, c^{lo} は 16 bit.

どちらも FP32 で無誤差に表せるので，
変換は 2^{31} まで無誤差になる.

⇒ 本稿の全 K で GEMMul8 と同じ
8 bit の法テーブルが使える.

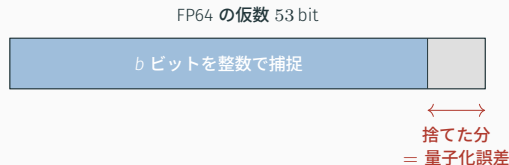
剰余は $d - \text{round}(d/m) m$ の 1 手で取る.
正しい代表元を返すのは $|d| < 2^{22}$ のとき
(実測でも全法で $2^{22.02}$ が最初の破綻点).

スキーム II の実装 (2) : 幅 b が精度を決める

オペランドを 2^b 倍して整数にするときの、
無誤差に捕捉するビット幅が b である：

$$A' = \text{round}(x/\tau \cdot 2^b)$$

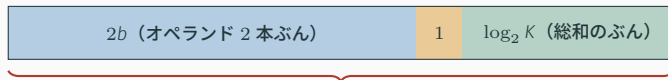
- ・ 捕捉しきれない分は量子化誤差 $|\delta| \leq 1/2$ として残る
- ・ $b=24$ が FP32 相当, FP64 相当には $b \approx 53$
- ・ scheme I のスライス基数 β とは別物 (β は 1 桁の大きさ, b は全体で何ビット取るか)



本実装の既定は $b=54$.
なぜ 53 でも 60 でもなく 54 なのかが、このあとの 3 枚の主題である.

b は利用者が与えるのではなく，CRT の窓から決まる

$$|C'| \leq K \cdot 2^{2b} < \mathcal{M}/2 \quad \Longleftrightarrow \quad \log_2 \mathcal{M} > 2b + 1 + \log_2 K$$



法の積 $\log_2 \mathcal{M}$ が，これを上回らなければならない

決まる向き

法の本数 s (=コスト) $\rightarrow \mathcal{M} \rightarrow b$

$$b = \left\lfloor \frac{\log_2 \mathcal{M} - 1 - \log_2 K}{2} \right\rfloor$$

$b=54$, $K=14080$ なら

窓は ≈ 123 bit 要る.

8 bit の法では $s=16$ 本.

これが「36 本 $\rightarrow$ 16 本」の 16 の出どころである.

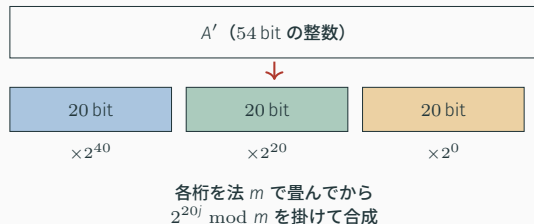
54 bit の整数は、1 つの FP32 に載らない

FP32 が無誤差に表せる整数は 2^{24} まで。

A' は 54 bit ある。

対処：スライサが A' を 2^{20} 進の n_d 桁として作り、各桁の剰余を $2^{20j} \bmod m$ で合成する。

- ・ 桁を先に法で畳んでから重みを掛けるので、積は $m^2/2$ 未満に収まり FP32 で無誤差
- ・ 桁数は b から決まる： $b \leq 60$ なら 3 桁、超えると 4 桁
- ・ 桁はスライサ内部の量なので、GEMM 本数もメモリも増えない



剰余は $d - \text{round}(d/m)m$ の 1 手で取る。
正しい代表元を返すのは $|d| < 2^{22}$ のとき（実測でも全法で $2^{22.02}$ が最初の破綻点）で、桁も総和もこの範囲に収まる。

法の積の過剰は、安全余裕ではなく**精度の損失**

再構成が解くのは分数 $\text{frac} = C' / M$ である。

そして triple-float はこれを**固定の絶対精度** ϵ_f で解く。

$\Rightarrow M$ が必要量を超えるほど **frac が小さくなり**、同じ**絶対精度でも相対精度が落ちる**。

窓びったり  $\longrightarrow$ **frac が大きい**

窓が過剰  $\longrightarrow$ **frac が小さい**

—— ϵ_f (どちらも同じ幅)

同じ ϵ_f でも、frac が小さいほど
相対誤差は大きくなる

実測 (K=7040, s は 16 のまま)

余剰 1 bit につき、要素ごとの

最大相対誤差が約 1 bit 悪化

($4.6 \times 10^{-16} \rightarrow 8.1 \times 10^{-15}$, 余剰 4 bit)。

ノルム比も余剰 8 bit で **54.2 $\rightarrow$ 51.3 bit**。

これは Blackhole 固有の問題である

差は**再構成の解き方**にある。GEMMu8 は CRT を FP64 の相対精度で解くので、 M が大きくても frac の有効桁は落ちない。

本実装は FP32 3 本組で固定の絶対精度 (2^{-99}) を持たせるため、 M が過剰なほど frac が小さくなり、**相対精度がその分失われる**。

M を「狭義最小」に詰める

$2b+1+\lceil\log_2 K\rceil$ を狭義に満たす最小の M を採る.

- ・ 末尾 2 個を, 必要量を最小の超過で満たす互いに素な対に選び直す
(1 個だけ選び直しても, 法 1 個ぶん約 8 bit の余りが残ってしまう)
- ・ 窓を満たしたまま落とせる法は落とす
8 bit の法では余りが出ず $s=16$ のままだが, 法が 10 bit ある FP8 経路では 16 本の指定に対し実際に使うのは 13-14 本

なぜ「狭義」でなければならないか

$|C'|=M/2$ ちょうどだと,
 $+M/2$ と $-M/2$ がすべての法で同じ剰余を与える.

⇒ CRT が両者を区別できず, 復元が一意でなくなる.

既定 $b=54$ の正体

窓の式は $s=16$ で $b=55-56$ を許す. それでも 54 で頭打ちにするのは同じ理由で, 欲張って窓を埋め切ると M が増えて再構成の相対精度が落ちるからである.

$b=54$ は「無損失」ではなく, FP64 相当に十分な水準で意図的に打ち切った値である.

b が大域の単一パラメタであることの副作用

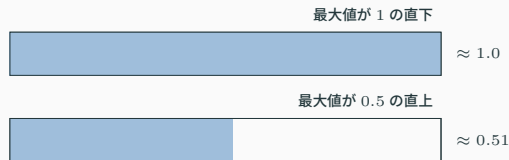
b は行列全体で 1 つの値である．一方スケール τ は 2 のべきへ切り上げて取る．

⇒ 正規化後の最大値は $(0.5, 1]$ のどこかに来る．

1 の直下でない入力では，捕捉できるビットがその分減る．

実測

各行・各列を無作為な実数倍した入力では
 $K=7040$ で $54.2 \rightarrow 53.9$ bit.



青が実際に使われるレンジ．
ほぼ 1 bit ぶん取りこぼす

入力の指数部レンジが広がると，この効き方はさらに大きくなる．
そこで必要な b を入力から決める自動決定を持たせた——第 5 部で述べる．

スキーム II の実装 (3) : FP8 経路の桁分割

FP8 が無誤差表現する整数は $[-16, 16]$ のみ.

素朴には法が ≤ 31 で、互いに素な奇数は高々 10 個

⇒ 到達精度が $b \approx 14$ (半精度相当) 止まり.

対処: 剰余を桁に分けて運ぶ.

対称剰余のまま 2 桁に分けると桁の上限が半分になり,

法を $m \leq 1089$ まで大きく取れる.

法 1 本あたり GEMM 3 本に抑える 2 方式:

- 平方数 $m=q^2$: 平衡 2 桁. $q^2 \equiv 0$ より最上位項が丸ごと消えて 3 本
- 非平方数: 基数 16 の Karatsuba. 桁を切り上げて分けて両桁を必ず逆符号にし, 和まで ± 16 に収める

効果

法が 8 bit から 10 bit へ広がるぶん本数も減り,
 14080^3 で法 14 本・GEMM 42 本.

法を 8 bit のまま平衡 2 桁で運ぶ構成 (法 16 本・GEMM 64 本) より 34% 少ない.

本経路の存在意義

桁が ≤ 16 (5 bit) で LoFi 乗算器 (SrcA 5 bit) に収まるので, その GEMM は LoFi で無誤差に走らせられる.

8 bit 剰余ゆえ HiFi4 を要する INT8 版とは対照的.

先行実装との差異（要点）

スキーム I（対 ozIMMU / OzBLAS）

- ・ **無誤差条件をどの自由度で解くか**：先行はスライス幅 w を K の関数に．本実装は w を固定し K の側を分割（ K から基数を決める案は $13\times$ 遅い）
- ・ **対角ごとのまとめ方**：本実装は K 連結で発行を $L+1$ 本へ（先行は $p(p+1)/2$ 本のまま）
- ・ **停止則**：残差 0 ではなくデータ依存の誤差限界
- ・ **再構成**：FP64 でなく triple-float の Horner

スキーム II（対 GEMMu8）

- ・ **CRT の解き方**：FP64 / double-double でなく **分數形を FP32 3 本組で**
- ・ **法の本数の詰め方**： M の過剰が精度損失になるので窓を満たす最小に詰める（FP64 で解く GEMMu8 には対応物なし）
- ・ **幅 b の自動決定**：量子化誤差と出力のノルム比から必要な b を求める
我々の知る限り scheme II でこれを持つ実装は他に無い
- ・ **法の使える範囲**：符号-絶対値 UInt8 ゆえ 256 が使えず $254=2 \cdot 127$ を置く

差異の多くは「FP64 を持たない」「SFPU が相対的に弱い」というハードウェアの性質から導かれている。

第 5 部：評価

性能

- ・ 有効演算量 $2MNK$ (分割回数に依らず通常の GEMM と同一) を実時間で割った値
- ・ 時間はデバイス常駐 DRAM→DRAM の 1 回呼び出し実測. 全工程をデバイス上で実行し, A, B, C の PCIe 転送は含まない
- ・ ウォームアップ 1 回の後, 3 回の算術平均 (最良値ではない)

独立測定の間では同一条件でも数% 変動する.

精度

- ・ ホスト上の FP64 補償付き総和 (TwoProd/TwoSum) による参照解に対する Frobenius ノルム相対誤差を実効 bit で
- ・ 参照は相対 2^{-87} 以上の精度 (128 bit 演算と突き合わせ確認)
- ・ 補助的に要素ごとの最大相対誤差も併記

入力

$$a_{ij} = (\text{rand} - 0.5) \exp(\varphi \cdot \text{randn})$$

既定は $\varphi=0$ (一様乱数). φ を振った評価は後述.

全方式の統一比較 (14080³, 110 コア)

方式	到達精度 [bit]	N_{pp}	実測 [TFlops/s]	モデル上限
BF16 最適化	~ 8 (限界)	1	117.0	117.0
BF16 extended	~ 13 (限界)	3	39.1	39.1
Ozaki-I-BF16	53.8	66	1.19	2.15
Ozaki-I-BFP8	53.8	66	1.41	2.99
Ozaki-I-INT8	54.2	36	2.10	2.74
Ozaki-I-FP8	53.9	105	1.38	2.12
Ozaki-II-INT8	54.2	16	2.73	5.39
Ozaki-II-FP8	54.2	42	2.08	3.63

N_{pp} = 部分積数 (scheme I は $p(p+1)/2$, scheme II-INT8 は法の本数 s). 「モデル上限」 = $2MNK/\text{GEMM 実行時間}$ (GEMM 段が律速する場合の上限).

- 通常 BF16 と BF16 extended はハードウェア側の精度限界で FP64 に届かない. 尾崎方式は整数無誤差累算により FP64 相当 (53-54 bit) に到達
- FP64 相当で揃えると Ozaki-II-INT8 が最速 (36 → 16 本の効果). scheme I の中では INT8 が最速 (radix が大きい + K ブロック化が不要)

なぜこの順位になるか

Ozaki-I-INT8 > Ozaki-I-FP8 の理由

部分積数では FP8 が不利 (105 対 36) だが、LoFi の 4 倍レートで走るので本数とレートだけなら FP8 が速いはず。

差を作るのは累算器の幅：

$b_{\text{acc}}=24$ ゆえ FP8 は K ブロック化が要り、再構成をブロック数だけ繰り返す。

Ozaki-I-BF16 が最下位の理由

再構成の反復が最も多いことに加え、GEMM 段自体の効率も低い。

14080³ の GEMM 段：

BF16 3935 ms • 93.6 TOps/s

FP8 3285 ms • 178.5 TOps/s (LoFi の高レート)

モデル上限に届かない理由

実測は GEMM 律速上限 ($2MNK/\text{GEMM 実行時間}$) の 53-77%。

差はスライス分解・再構成という非 GEMM 周辺工程である。 K ブロック化の要らない INT8 経路では FLOP 数に依らない $O(MK+KN+MN)$ の固定費として残り、ブロック化する BF16 / FP8 経路では再構成がブロック数だけ繰り返される。

工程別の実行時間 (14080³, 110 コア)

経路	slice	時間 [ms]			slice	割合 [%]	
		GEMM	再構成	合計		GEMM	再構成
Ozaki-I-BF16	365	2703	1034	4103	9	66	25
Ozaki-I-BFP8	365	1938	1046	3350	11	58	31
Ozaki-I-FP8	447	2546	979	3972	11	64	25
Ozaki-I-INT8	335	2043	286	2663	13	77	11
Ozaki-II-INT8	412	1035	595	2041	20	51	29
Ozaki-II-FP8	386	1715	595	2696	14	64	22

読み方

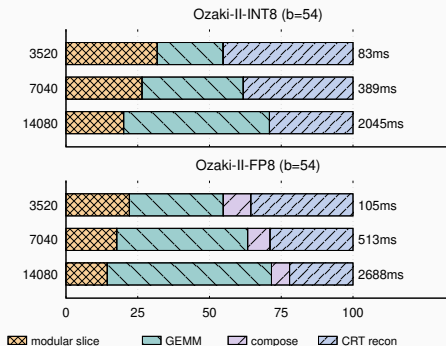
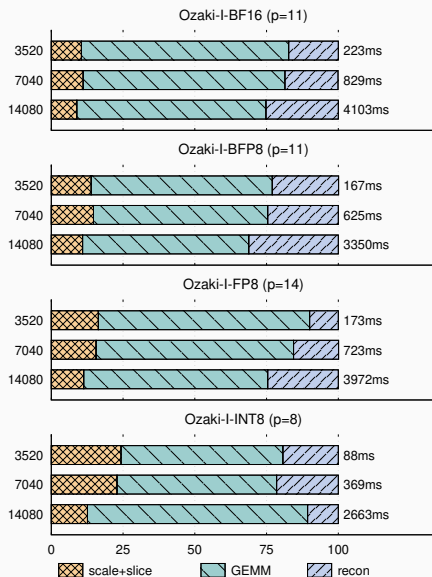
K ブロック化の要らない INT8 だけが GEMM 支配 (77%) .

逆に最も再構成へ寄るのは BFP8 (31%) ——GEMM を速くしたぶん比重が上がる.

scheme II は GEMM 本数を 36 → 16 へ減らしたぶん 非 GEMM の比重が大きい (slice 20%) .

Ozaki-II-FP8 の GEMM は桁積の合成段を含む. BF16 / BFP8 / FP8 の再構成は遅延再構成を用いた値.

実行時間の内訳



scheme II はサイズで律速要因が移る

scheme I は GEMM 支配

scheme I は GEMM 支配

GEMM が $O(N^3)$ なのに対し scale・スライス分解は $O(N^2)$ の低次工程であるため。

ただし**比率はサイズとともに単調には上がらない**： $b_{acc}=24$ の BF16 / FP8 経路は N に比例して K ブロック数が増え、**再構成をその回数だけ繰り返す**。

⇒ K ブロック化の要らない INT8 だけが**最大サイズで最も GEMM 支配**へ寄る。

scheme II はサイズで律速要因が移る

GEMM 本数を $36 \rightarrow 16$ へ減らしたぶん**非 GEMM 工程の比重が scheme I より大きい**。

INT8 版は**最小サイズでは再構成が最大の段**だが、GEMM だけが $O(N^3)$ で伸びるので最大サイズでは GEMM 支配へ転じる。

FP8 版は桁積の合成段を持つぶん再構成段が軽いが、桁分割で GEMM を 42 本へ増やすため、LoFi で走らせても **GEMM 段は INT8 版の ≈ 1.6 倍に膨らむ**。

なぜ再構成がこれほど重いのか

(1) 演算器が桁違いに弱い

再構成は行列エンジンではなく SFPU で走る．4 経路の実測時間を SFPU タイル演算数で割ると 3.3-5.3 要素/サイクル/コア—行列エンジンの 2048 積和/サイクル/コア (LoFi) の 1/400 程度である．

DRAM 往復は 14080^3 の 10 ブロックで 135 GB (500 GB/s なら 0.27 s) にすぎない．実測は 1.75 s なので、この段は明確に SFPU 律速であって帯域ではない．

(2) triple-float に FMA が無く、1 段が重い

SFPU のタイル演算 API は加減算・乗算だけで FMA もレジスタ間の move も無い．無誤差な積には Dekker 分割が要り，Horner の 1 段 ($\text{acc} \leftarrow \text{acc} \cdot \beta + P_d$) は tfMulR 46 演算 + tfAddS 17 演算 = 64 演算になる．

(3) K ブロック数だけ繰り返す

再構成は K ブロックごとに出力全体をなめ直す $\Rightarrow$ 総演算量 $\propto K_{\text{blk}} \times n_{\text{diag}}$ ．

経路間の 6 倍差は、ほぼ K ブロック数の差

経路	K blk	n_{diag}	演算 /タイル	総演算 [10^9]	時間 [ms]	実効 [要素/cyc]
Ozaki-I-BF16	10	11	643	1.34	1740	5.3
Ozaki-I-BFP8	10	11	643	1.34	1753	5.3
Ozaki-I-FP8	4	14	835	0.68	982	4.8
Ozaki-I-INT8	1	8	714	0.14	290	3.3

根はひとつ：累算器幅 b_{acc}

無誤差条件 $2w + \lceil \log_2 K_{\text{eff}} \rceil \leq b_{\text{acc}}$:

INT32 ($b_{\text{acc}}=31$) なら $14+17=31$ でちょうど収まる $\Rightarrow K$ ブロック化なし $\Rightarrow$ 再構成 1 回

FP32 ($b_{\text{acc}}=24$) は $10+18=28 > 24 \Rightarrow$ 10 分割 $\Rightarrow$ 再構成 10 回

同じ 24 bit が、GEMM 側では基数を 31 に縛り、再構成側では回数を 10 倍にする。

遅延再構成：K ブロックごとに Horner を回さない

対角ごとの部分和を**整数のまま** K 方向に累積し、最後に 1 回だけ Horner

Horner は P_d について線形なので結果は変わらない。全 K ブロックを合計しても 1 語に収まる：

$$|S_d| \leq n_{kb}(d+1)K_c\beta^2 = 10 \cdot 11 \cdot 1408 \cdot 31^2 = 1.49 \times 10^8 < 2^{31} \quad (14 \text{ 倍の余裕})$$

経路	再構成	GEMM	再構成 [ms]	合計	実効 [TFlops/s]
Ozaki-I-BF16	ブロック毎	2554	1732	4651	1.200
	遅延	2703	1034	4103	1.361
Ozaki-I-BFP8	ブロック毎	1821	1746	3932	1.420
	遅延	1938	1046	3350	1.666

代償は二つ

- 累積バッファは P_{big} とは別に n_{diag} 面要る (8.7 GB 増)。対角 14 面の FP8 は 14080^3 で入らず従来経路へ退避。
- バッファが増えて DRAM の割り付けが変わり GEMM 段が 5-6% 落ちる。

到達精度は両経路でビット単位に一致 (K ブロック 4 個の $1408 \times 1408 \times 5632$ で確認)。

実装上の罫：DST 上の Int32 の表現が経路で違う

符号-絶対値と 2 の補数が混在する

行列エンジンがパックした Int32 を copy_tile で載せる $\Rightarrow$ 符号-絶対値
(INT8 経路の復号はマスク $0x7FFFFFF00$ + 符号復元を前提にしている)

typecast<FP32, Int32> が作る $\Rightarrow$ 2 の補数
(add_int_tile も Blackhole では 2 の補数前提)

気づかずに既存の復号を流用して、相対誤差 6.4×10^4 で壊れた.

2 の補数なら符号の復元が要らない

$$v = (v \& 0xFFFFFFFF00) + (v \& 0xFF)$$

上位は 256 の倍数で $|v| < 2^{28}$ なら有効桁 20 bit 以下, 下位は 255 以下——どちらも FP32 が無誤差に表せる. 符号-絶対値版より 5 演算少ない.

未実装の余地: 基数が $2^k - 1$ なら Dekker 分割も避けられる ($a=32x$, $p=a-x$, $z=a-p$, $e=z-x$ で $31x=p+e$ が無誤差).

サイズ・スケーリング (実効 [TFlops/s])

経路	3520 ³	7040 ³	14080 ³
BF16 最適化 (～ 8 bit)	107.3	117.2	122.8
BF16 extended (～ 13 bit)	34.3	37.4	39.0
Ozaki-I-BF16	0.391	0.842	1.361
Ozaki-I-BFP8	0.521	1.116	1.666
Ozaki-I-FP8	0.506	0.966	1.405
Ozaki-I-INT8	0.988	1.893	2.096
Ozaki-II-INT8	1.059	1.788	2.735
Ozaki-II-FP8	0.836	1.359	2.070

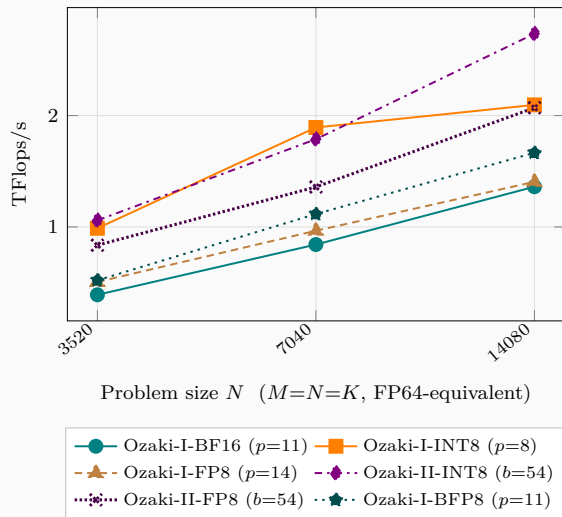
両端は動かない

最上位は Ozaki-II-INT8, 最下位は Ozaki-I-BF16 (14080³ で 2.74 対 1.36)。

7040³ だけ順位が入れ替わる

Ozaki-I-INT8 が 1.89 対 1.79 で上回る——この寸法で scheme I の GEMM 段が天井の 77% と本稿で最も効率に達し、総時間の 56% を占めるのに対し, scheme II はまだ 35% で非 GEMM が効く。

サイズ・スケーリング



- 全方式とも N とともに単調に伸びる
(コアあたり出力ブロックが大きくなり GEMM の演算強度が上がる)
- 両端は全サイズで不変 (最上位 Ozaki-II-INT8, 最下位 Ozaki-I-BF16).
中位は 3520^3 のみ I-INT8 と II-FP8 がばらつきの範囲で並ぶ (0.82 対 0.83)
- 差が最も詰まるのは 7040^3 (9% 差):
scheme I の GEMM 段が効率に達して総時間の 62% を占める一方, scheme II はまだ 35%
- 14080^3 では scheme II も GEMM が 53% を占め,
本数 16 対 36 の差がそのまま 26% の差に

サイズの選び方: 110 コアを均等に使うには $N/32$ が 110 の倍数.

さらに 2 の因子に富むほどタイリングが良い

計算可能サイズの上限：DRAM ではなく L1

評価可能な K を律速するのは **per-core L1 容量** (1.5 MiB/コア).

律速する CB は GEMM でも DST でもなく,
スライス段の最大値リダクション用ステージング：
 K ベクトル全体を BF16 で保持し, $2048(K/32+2)$ B/コア.

Ozaki-I-INT8 / BF16 で **バイト単位で同一**かつ p 非依存.
両経路とも $K=16384$ 直下が上限.

$\Rightarrow 14080^3$ が L1 上限に収まる最大の 110 コア整合サイズ.
次の候補 17600 は超過して計算できない.

原理的には除ける

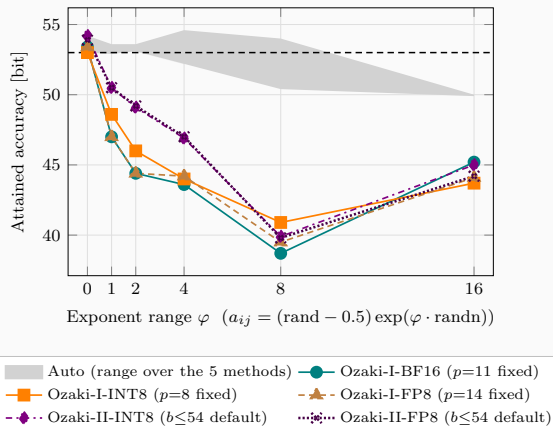
内積 (K) 方向のブロッキング—
最大値リダクション・スライス・GEMM・累積を**ブロック毎に流し**, L1 常駐量を K 非依存に抑える.

ただし現状は未対応

K ブロック化 (課題 2) は GEMM と再構成を**ブロック毎に流すが**, スケール最大値リダクションとスライス分解は依然として K 全体を一度に処理するので, この上限には効かない.

両段のブロック化は今後の課題.

入力の指数部レンジへの依存性



尾崎スキームの分解はデータ依存である：
行内の最大値から大きく下回る要素ほど多くの情報量を要する。

- 折れ線=パラメタを固定したまま：
 $\varphi=8$ で最大 14 bit 落ちる
(方式の破綻ではなくスライス数の不足)
- 帯=自動決定を効かせたとき：
 $\varphi \leq 8$ で 50-55 bit を保つ

代償は時間：INT8 の $p=8 \rightarrow 10$ は部分積
36 $\rightarrow$ 55 本。
14080³ の実測でも $p=9$ で 2.05 $\rightarrow$ 1.71 TFlops/s
(17% 落ち)。

scheme II 側：量子化誤差モデルと Hutchinson
スケッチで $\|C\|_F$ を測り、必要な b を決める
($b=53-69$)。

$\varphi=16$ ：どこで止まるか

自動決定を効かせても 5 方式とも 49.9–50.0 bit で FP64 相当に届かない。

ただし律速は b でも法の本数でもない：

- b を $60 \rightarrow 71$ に上げてても $49.5 \rightarrow 50.0$ bit
- b と法を揃えたまま再構成を 1 パスへ戻すと 32.4 bit へ落ちる
(2 パス化 = $1/m_i$ を共通粒度へ丸めて第 1 パスの累算を無誤差にする手当て。予備スライド参照)

効いているのは frac を解く絶対精度 ϵ_f ：

$$\text{ノルム比} = \epsilon_f Q, \quad Q = \mathcal{M} 2^{-2b} \frac{\|\tau^A\| \|\tau^B\|}{\|C\|_F}$$

$Q=2^{48.7}$ から $\epsilon_f=2^{-98.7}$ が逆算でき、53 bit には $\epsilon_f \leq 2^{-102}$ が要る。

原理的には対応できる

ϵ_f を細かくする以外に Q を下げる道がある。

絶対値上界行列どうしの行列積を 1 本追加して窓を実測すれば、 $\varphi \geq 4$ では Q を 12 bit 詰められる (行の最大値と列の最大値が別の k に居るため)。

追加コストは $s=20$ 本に対し 5%。

本稿はいずれも実装しておらず、最悪ケース上界のままの値を報告している。

同じ要求は CRT を FP64 で解く GEMMul8 にもかかる (同実装の CRT 定数は実効 95 bit 程度)。
これは CRT 定数そのものの絶対精度の話で、前述の $\mathcal{M}$ の過剰とは別の軸である。

第 6 部：考察とまとめ

1. 無誤差分割 (BF16×2) は本ハードウェアでは有効でない

GPU の TF32 分割 (3xTF32) は積を完全精度で生成し真の FP32 で累算するので同型分割で FP32 を回復できる。

Blackhole の FP 累算経路はブロック浮動小数点により実効 ≈ 13 bit へ縮約されるため、項を増やしても伸びない。

BF16×2 は出力パネル分割で 14080^3 まで動くので、制約は容量ではなく精度側にある。

2. 性能ボトルネックは無誤差分解の構造的オーバーヘッド

行列エンジンは、入力が実際に供給できる範囲では高効率 (INT8 経路は HiFi4 ピークの 63%, BF16 経路はパス数以外の天井 175 に対し 53%)。

乖離の要因は 2 つ：無誤差条件が課す構造的な部分積数と、SFPU 上の非 GEMM 周辺工程。

GEMM 本数を $36 \rightarrow 16$ に減らす scheme II でも、増えた非 GEMM 工程で利得の大半が失われ、本数比が示す $2.3\times$ に対し実測は $1.26\times$ にとどまる。

天井の正体：アンパッカが送れる量

一次資料が与える 2 つの数から、天井が計算で出る。

- ・ 32×32 タイル 1 枚の積は **LoFi 16 サイクル** (HiFin で $16n$)
- ・ アンパッカが L1 から SrcA・SrcB へ送れるのは **両者あわせて 80 B/サイクル**

タイル 1 枚が消費するのは $A \cdot B$ 各 1 タイル = 2048 要素なので、要素幅 W バイトに対し必要な供給は $2048W/16n$ B/サイクル。

LoFi が全速で回れるのは $W \leq 0.625 B = 5 \text{ bit}$ のときだけ

これは **SrcA の乗算器幅と一致する**——この機械は LoFi で 5 bit のオペランドに釣り合っている。

	要素幅	LoFi で必要	上限 [TFlops/s]
BF16 (本実装)	2 B	256 B/cyc	190
BFP8_b	1.0625 B	136 B/cyc	358
FP8 / INT8	1 B	128 B/cyc	380
BFP4_b	0.5625 B	72 B/cyc	608 (天井なし)

実測の頭打ちは N とともに $142.4 \rightarrow 162.3 \rightarrow 175.0$ とこの 190 へ近づく (14080^3 で 92%)。

最適化した BF16 GEMM を 3 サイズ × 4 fidelity で測る（立方，110 コア，FP32 累算）：

fid	実測 [TFlops/s]			天井	14080 ³ の内訳	
	3520	7040	14080		達成	$e(K)$ 予測
LoFi	142.8	162.4	174.9	190.1	92%	174.3
HiFi2	142.8	162.2	175.0	190.1	92%	174.3
HiFi3	122.8	136.3	144.9	190.1	76%	—
HiFi4	102.8	112.0	117.2	152.1	77%	—

供給律速の 2 段は，天井 × $e(K)$ でほぼ決まる

$e(K)=1/(1+1274/K)$ の予測は 139.6 / 161.0 / 174.3，実測との差は 2.3 / 0.9 / 0.3%.

⇒ 素の GEMM の効率の落ちは 1 本あたりの固定費（ $K=1274$ 相当）だけで説明がつく．

演算律速の HiFi3 / HiFi4 は 76–77% で， $e(K)$ では説明できない（本稿では未同定）．

天井に届くには、コアあたりブロックの大きさが要る

$K=14080 \cdot \text{LoFi}$ に固定し、 M, N を変えて **コアあたり** 出力ブロックだけを振る（**コア格子は全行とも $10 \times 11 = 110$** 。110 コアは常に使い切っている）：

コアあたり $p_m \times p_n$	再利用比 $\frac{p_m p_n}{p_m + p_n}$	$M \times N$	使用 コア	実測	予測	比
2×2	1.00	640×704	110	44.5	44.4	1.00
4×4	2.00	1280×1408	110	90.9	88.7	1.03
4×8	2.67	1280×2816	110	102.5	118.3	0.87
8×4	2.67	2560×1408	110	106.3	118.3	0.90
8×8	4.00	2560×2816	110	177.4	177.4	1.00
11×10	5.24	3520×3520	110	175.0	232.3	0.75

再利用比に比例し、 8×8 で頭打ちになる

$2 \times 2 \rightarrow 8 \times 8$ で再利用比が 4 倍 $\Rightarrow$ 実測も $44.5 \rightarrow 177.4$ と 4.0 倍。11×10（再利用比 5.24）は 175.0 で 8×8 を上回らない——供給天井 190.1 に当たる。

$\Rightarrow$ 190.1 という天井は 8×8 以上でしか意味を持たない。

ただし機構は同定できていない。計算カーネルはタイル積ごとに `matmul_tiles` を呼び `ct_dim/rt_dim` は 1 なので、アンパッカから見れば形状に依らないはず。DRAM 入力帯域が一定というモデルでも等しく説明がつく（ $MN/(M+N)$ で割

ると正方 3 点が 132.8 / 135.6 / 132.3 GB/s に揃う）。本稿の測定点では両者を区別できない。

6 経路の性能と GEMM 段の効率 (14080³, 110 コア)

経路	fid	N_{pp}	K	形状	bw	GEMM 段		達成	上限	実効
			blk			[TOps/s]	天井			
Ozaki-I-BF16	LoFi	66	10	8×8	11	136.3	190.1	72%	2.07	1.361
Ozaki-I-BFP8	LoFi	66	10	8×8	22	190.1	357.8	53%	2.88	1.666
Ozaki-I-FP8	LoFi	105	4	8×8	22	230.2	380.2	61%	2.19	1.405
Ozaki-I-INT8	HiFi4	36	1	4×4	55	98.4	152.1	65%	2.73	2.096
Ozaki-II-INT8	HiFi4	16	—	11×10	—	86.0	152.1	57%	5.38	2.735
Ozaki-II-FP8	LoFi	42	—	11×10	—	152.5	380.2	40%	3.63	2.070

達成率の低さと絶対値の低さは別

BF16 の 72% は 6 経路で最高だが、天井が 190 と最低なので絶対値は 136. BFP8 は 53% でも天井 358 で 190 出る. 天井を上げた経路ほど距離が開いて見える.

なぜ GEMM 段は天井に届かないのか (Ozaki-I-BF16)

素の GEMM から出発して、条件を一つずつ尾崎の GEMM 段へ近づけて測る：

条件	[TOps/s]	天井比	差分と、その正体
供給天井 (8×8 以上)	190.1	100%	—
素の GEMM, 8×8, $K=14080$	177.4	93%	−7: 1 本あたりの固定費 $e(K)$
対角 11 本に割る	167.3	88%	−5: K が不均一 ($d=0$ は 99.4)
端数パネル込み	157.3	83%	−5: 9% の端数が 4×8 へ落ちる
実測 (Ozaki-I-BF16)	143.9	76%	−7: 尾崎固有

各行は素の BF16 GEMM を対応する形状・ K で実行した実測値（最終行のみ尾崎）。複数本の合計は仕事量で重み付け。最終行は遅延再構成を切った構成（累積バッファを持つと DRAM の割り付けが変わって GEMM 段が 5% 落ちるので、GEMM 段自体の分解には含めない）。

同形状の素 GEMM と比べ直す

公称天井 $\min(608.3/n, 380.2/W)$ は 8×8 以上を前提にした値である．そこで各経路と同じ形状・同じ fidelity で走らせた素の BF16 GEMM を分母に取り直す：

経路	fid	形状	再利用比	公称天井	同形状の素	実測 [TOps/s] 値	素比
Ozaki-I-BF16	LoFi	8×8	4.00	190.1	177.4	143.9	81%
Ozaki-I-BFP8	LoFi	8×8	4.00	357.8	—	201.8	—
Ozaki-I-FP8	LoFi	8×8	4.00	380.2	—	230.2	—
Ozaki-I-INT8	HiFi4	4×4	2.00	152.1	91.0	98.1	108%
Ozaki-II-INT8	HiFi4	11×10	5.24	152.1	117.0	86.0	74%
Ozaki-II-FP8	LoFi	11×10	5.24	380.2	175.0	152.5	87%

ただし INT8 では形状そのものが効いていない

コアあたり形状を強制して 14080^3 を測ると 4×4 104.7 / 8×4 103.9 / 4×8 101.8 / 8×8 100.4 TOps/s
——再利用比を倍にしてもむしろわずかに遅い．

素の BF16 が 91 前後に張り付くのは入力 が 2 バイトで DRAM 側が先に尽きるためで、1 バイトの INT8 にその制約はない．

⇒ 入力形式が違う経路では、この比較（素比 108%）は成り立たない．INT8 を 100–105 に留めている要因は同定できていない．

形状を決めているのは L1

bw と形状は，同じ 1.25 MiB を奪い合う

尾崎の GEMM は無誤差累積のため出力側を FP32 / INT32 (4096 B/タイル) で持ち，さらに K サブブロック間のスピル用に c_{24} を抱える．Ozaki-I-INT8 は $bw=55$ を取った代わりに形状が 4×4 になっている——前ページの掃引が示すとおり，この経路ではこの選択に損はない．

未実装の余地： c_{16} と c_{24} の L1 共有

本実装は 2 つを別々に確保しているが，tt-metalium の標準カーネルは L1 領域を共有させている．共有すれば出力側の常駐が 1 面ぶん減る ($11\times 10 \cdot \text{INT32}$ なら 440 KiB/コア)．

ただし INT8 経路では形状も bw も効いていないので，
この改修がどの経路に効くかは測ってみないと分からない．

GEMM 段は改善できるか

−5（対角）：スキーム I の定義そのもの

K 連結で対角ごとに $K=(d+1)K_c$ と不均一になる．短い側 ($d=0$, $K=1408$) は天井の半分しか出ない．アルゴリズムを変えない限り取り返せない．

−5：問題サイズ依存

$M_t=440$ がパネル高 80 で割り切れず，端数の 5 枚がコアあたり 4×8 へ落ちる（再利用比 $p_m p_n / (p_m + p_n)$ が $4 \rightarrow 2.67$)． M_t が倍数なら消えるが，任意サイズでは残る．

−7：尾崎固有．ここだけが実装の余地

測れたのはホスト側のプログラム発行で，GEMM 段 1863 ms 中 42.6 ms (2.3%) ——発行回数は主因ではない．デバイス側へ入れても 1 ポイント強．

残りは対角ごとのオフセット計算・出力 CB のスロット管理・ K スピル往復 (c_{24})．スピル用カーネルは尾崎側の CB を前提とするため素の GEMM へ差し替えられず，内訳は同定できていない．

天井との差 24 ポイント = 素の GEMM の固定費 7 + 構造的 10 + 実装の余地 7

形式を狭めた利得は、天井の比ほどには出ない

形状 (8×8) と bw を揃えて比べる：

形式	bw	GEMM 段 [TOps/s]	何が効いたか
BF16	11	141.8	(基準)
BFP8	11	167.5	形式だけで 1.18×
BFP8	22	196.2	bw が伸びて 1.17×

天井比は 1.88 なのに、形式そのものの寄与は 1.18

$t = t_{\text{供給}} + t_{\text{固定}}$ ($t_{\text{供給}} \propto W$) と置いて 2 点から解くと、 $t_{\text{固定}}$ が BF16 の 67%, BFP8 の 79%.

出力側が入力形式によらず FP32 のままだからである—— $C_{16} \cdot C_{24}$ は 4096 B/タイル, K サブブロックごとのスピル往復も FP32. 入力だけ半分にしてもそこは縮まない.

bw を倍にするとスピル往復が半減し、それが 2 番目の 1.17×.

Blackhole は LoFi で 580 TFlops/s（ピークの 86-96%）を出している

LoFi が 4 倍出ないのではない．条件が揃えば出る．

ただし一次資料がベンチマークする形式と fidelity の対応は：

組み合わせ	LoFi 換算の必要供給
BFLOAT16 → HiFi4	64 B/cyc
BFP8_b → HiFi2	64 B/cyc
BFP4_b → LoFi	64 B/cyc

すべて 80 のすぐ下，きっかり 64 B/サイクルに揃っている．
本研究が BF16 を LoFi で走らせると 256—4 倍の超過になる．

なお 80 B/サイクルは一次資料の性能モデル中の記述で，行列積に適用した記述は無い．

3. 低精度の高いレートは，無誤差条件下では実効性能に現れない

(i) 乗れないもの：INT8 経路は基数を 5 bit に落としても低 fidelity で答えが出ない． $\Rightarrow$ 4 倍差はそもそも生じない．

(ii) 乗れても 4 倍にはならないもの：BF16 ($\beta=31$) と FP8 ($\beta=15$) は LoFi で走るが，HiFi4 に固定した場合との差は 14080^3 で 12%， 7040^3 で 0.6% (0.93 対 0.83)．

素の BF16 GEMM を振ると (14080^3 ，FP32 累算) $175.0 / 174.8 / 144.7 / 117.1$ TFlops/s — LoFi と HiFi2 が同値で， ≈ 175 にパス数以外の天井がある．しかも尾崎の GEMM 段は 93.6 とその手前．

$\Rightarrow$ パス数が $1/4$ であること自体は 4 倍の実効性能を与えない．FP8 の GEMM 段は 178.5 TOps/s で HiFi4 のピーク 152 を超えるが，供給の軽さ・部分積数・K ブロック化の寄与は本稿では分離できていない．

4. 計算可能な K は DRAM ではなく per-core L1 容量に律速される

スライス段のステージング CB が K に比例して L1 を占める．

K 方向のブロッキングで原理的に除けるが未対応であり今後の課題．

AI アクセラレータが科学技術計算をも担うには：

1. 行列エンジンの累算経路に、ブロック浮動小数点を介さない無誤差な累算（真の FP32 または整数）を備えること
2. 低精度の高速モードでも乗算器のオペランド幅を確保し、無誤差スライスの基数と演算レートを両立させること
3. 多倍長再構成に効くプリミティブをベクタ演算器側に持つこと（FMA、無誤差変換に使える演算）

いずれも「精度を上げる」提言ではなく、「無誤差でいられる幅を確保する」提言である。尾崎スキームが要求するのは高精度演算器ではなく、丸めが起きない領域であるため。

Tenstorrent Blackhole 上で、科学技術計算が要求する高精度の密行列積をどこまで構成できるかを実機評価した。

- ・ FP32 相当を狙う無誤差分割は実効 ≈ 13 bit で頭打ちになり，その原因は入力の変数長ではなく FP 積・累算経路が無誤差に保持できる整数ビット幅というハードウェア側の限界にある。
- ・ これを回避するため尾崎スキームを完全オンデバイス実装した．スケール算出から再構成までデバイス上で実行し，到達精度がスライス数に応じて FP64 相当（実効 ≈ 53 bit）まで連続可変な行列積を得た．
- ・ 5 経路を実装・比較し， 14080^3 で 2.58 TFlops/s (Ozaki-II-INT8) が最速であった．
- ・ 障害は部分積数以上に 無誤差性を保てる経路が細いことにある．

本研究は JSPS 科研費 JP25K24387 の助成を受けたものである．

両実装に共通する最適化として、工程境界の二段階融合を行った（いずれもスケール／正規化が 2 の冪ゆえ lossless）：

- ・ Half 1：再構成出力へのスケール τ_m^A, τ_n^B 乗算を、最終の FP64 組み立てへ畳み込む（2 のべきなので指数への整数加算で済む）
⇒ デバイス上のスケール乗算 2 パス（ ≈ 0.31 ms）を消す
- ・ Half 2：per-tile 正規化を slice カーネルの第 0 反復に取り込む
⇒ 独立 dispatch と中間 DRAM バッファを消す

ハードウェア固有の制約

triple-float 残差（FP32 3 本）はスライス間で CB 経由に往復させる．

（一度に 3 タイルを渡すと循環バッファがデッドロックするため 2+1 タイルに分けて pack と copy-back）

宛先レジスタ DST の内容は per-slice の acquire/release を跨いで保持されないため、残差を DST に常駐させる実装は不正となり、採れない．

予備：scheme II の幅 b に対する到達精度

b	分割のみ（厳密演算）			実機（2 パス再構成）		
	$\varphi=0$	$\varphi=2$	$\varphi=4$	$\varphi=0$	$\varphi=2$	$\varphi=4$
48	48.5	43.9	40.3	48.5	43.7	41.2
52	52.8	47.6	45.5	52.7	47.7	45.6
56	∞	51.9	48.4	54.2	51.7	50.2
60	∞	56.1	53.3	54.2	54.1	53.7
64	∞	59.9	56.6	54.2	54.2	55.0
68	∞	65.2	60.1	54.2	54.2	55.0

- ・ 実機は $b \leq 56$ では厳密模擬を 1 bit 以内で追う
- ・ $b \geq 60$ では出力が FP64 であること自体が床になる
- ・ $\varphi=0$ の ∞ は b が FP64 の仮数を覆い切って無誤差になる意

予備：再構成の 2 パス化

FP32 3 本組の素の累算は最下位 limb の丸めで $2^{-66.7}$ に頭打ちになり、 b を上げて frac を細かくしてもそれを受け取れない。

粒度を揃えて累算を無誤差にする

$1/m_i$ を共通の絶対粒度 2^{-64} へ丸めた R_i と、切り捨てた残り $1/m_i - R_i$ の 2 つに分けて別々に積む。
 k_i が整数で R_i が共通粒度の整数倍なので、 $k_i R_i$ もその総和も同じ粒度の整数倍になり、
第 1 パスの累算は 3 本組の中で完全に無誤差に閉じる。

- ・ b と法を揃えたまま 1 パスと入れ替えると、 $\varphi=8 \cdot b=69$ で $43.0 \rightarrow 54.0$ bit の差
- ・ コストは再構成段が約 1.8 倍。 $\varphi=0$ では利得が無いため既定は 1 パス

GEMMu8 は CRT 定数を 2 本の double の非評価和で持ち、上位側を切り詰めて上位の累算を無誤差にしている（実効 95 bit 程度）。本実装の FP32 3 本組は公称 72 bit で、FP64 の 53 bit より多いどころか GEMMu8 より 20 bit 以上薄い。2 パス化はこの差を埋める手当てである。